

Shared Memory Programming with OpenMP

Pacheco chapter 5

Moreno Marzolla
Dip. di Informatica—Scienza e Ingegneria (DISI)
Università di Bologna

Copyright © 2013, 2014, 2017–2026
Moreno Marzolla, Università di Bologna, Italy
<https://www.moreno.marzolla.name/teaching/CP/>



Except where stated otherwise, this work is licensed under the Creative Commons Attribution-ShareAlike 4.0 International License (CC BY-SA 4.0). To view a copy of this license, visit <https://creativecommons.org/licenses/by-sa/4.0/> or send a letter to Creative Commons, 543 Howard Street, 5th Floor, San Francisco, California, 94105, USA.

Credits

- Peter Pacheco, Dept. of Computer Science, University of San Francisco
<http://www.cs.usfca.edu/~peter/>
- Mary Hall, School of Computing, University of Utah
<https://www.cs.utah.edu/~mhall/>
- Salvatore Orlando, DAIS, Università Ca' Foscari di Venezia
- Tim Mattson, Intel
- Blaise Barney, OpenMP
<https://hpc-tutorials.llnl.gov/openmp/> (highly recommended!!)

OpenMP

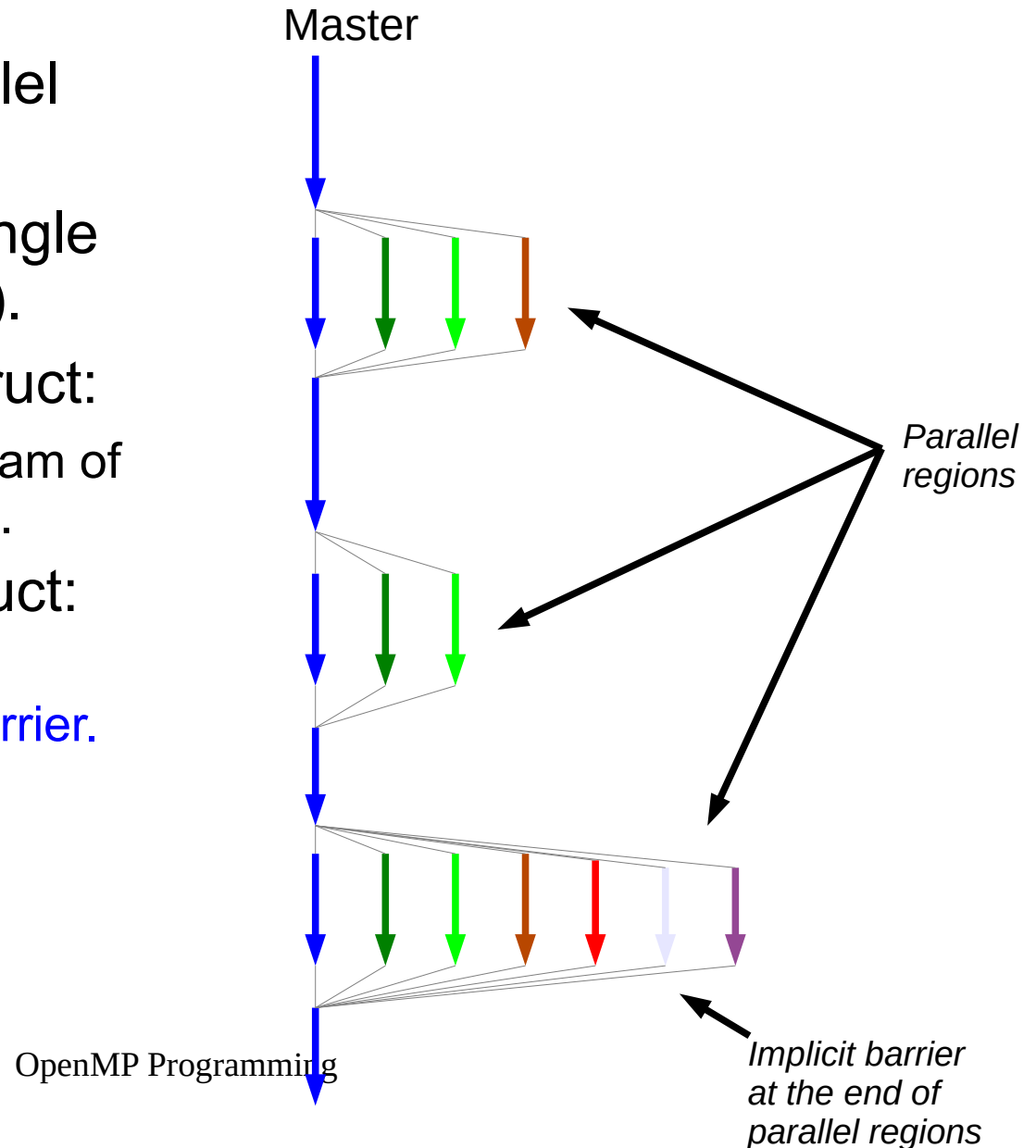
- Model for shared-memory parallel programming
- Portable across shared-memory architectures
- Incremental parallelization
 - Parallelize individual computations in a program while leaving the rest of the program sequential
- Compiler based
 - Compiler generates thread programs and synchronization
- Extensions to existing programming languages (Fortran, C and C++)
 - mainly by directives (`#pragma omp ...`)
 - a few library routines

A Programmer's View of OpenMP

- OpenMP is a portable, threaded, shared-memory programming specification with “light” syntax
 - Requires compiler support (C/C++ or Fortran)
- OpenMP will:
 - Allow a programmer to separate a program into serial regions and parallel regions
 - Provide synchronization constructs
- OpenMP will **not**:
 - Parallelize automatically
 - Guarantee speedup
 - Avoid data races

OpenMP Execution Model

- Fork-join model of parallel execution.
- Begin execution as a single process (master thread).
- Start of a parallel construct:
 - Master thread creates team of threads (worker threads).
- End of a parallel construct:
 - Threads in the team synchronize – **implicit barrier**.
- Only the master thread continues execution.



OpenMP uses Pragma

`#pragma omp construct [clause [clause ...]]`

- *Pragmas* are special preprocessor directives
 - They allow behaviors that are not part of the C specification
- Compilers that don't support the pragma ignore it
- Most OpenMP constructs apply to the **structured block** following the directive
 - Structured block: a block of one or more statements with **one point of entry** at the top and **one point of exit** at the bottom
 - Returning from inside a parallel block is not allowed

The `#pragma omp parallel` directive

- When a thread reaches a `parallel` directive, it creates a team of threads and becomes the master of the team.
 - The master has thread ID 0.
- The default team size is `implementation-dependent`.
- The code of the parallel region is duplicated and all threads will execute it.
- There is an `implied barrier at the end of a parallel section`. Only the master continues execution past this point.

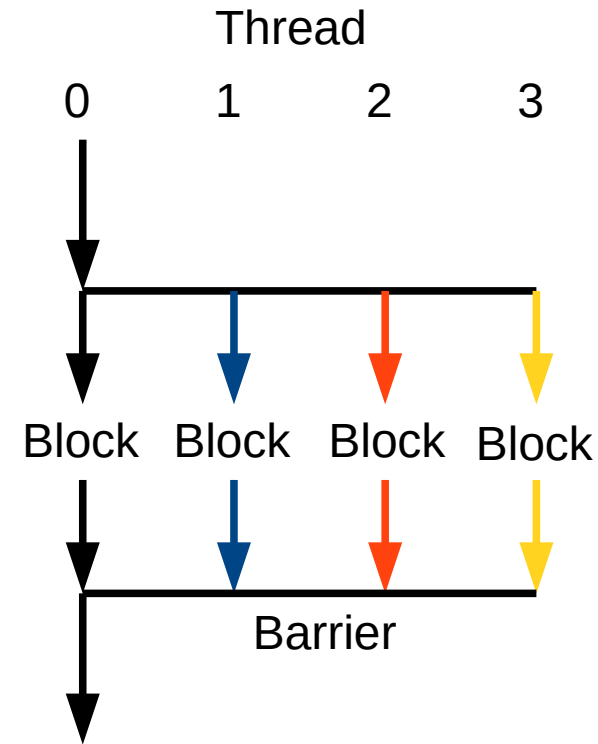
```
#pragma omp parallel [clause ...]  
clause ::=  
    if (scalar_expression) |  
    private (list) |  
    shared (list) |  
    default (shared | none) |  
    firstprivate (list) |  
    reduction (operator: list) |  
    copyin (list) |  
    num_threads (thr)
```


“Hello, world” in OpenMP

```
/* omp-demo0.c */
#include <stdio.h>

int main( void )
{
    #pragma omp parallel
    {
        printf("Hello, world!\n");
    }

    return 0;
}
```



```
$ gcc -fopenmp omp-demo0.c -o omp-demo0
$ ./omp-demo0
Hello, world!
Hello, world!
$ OMP_NUM_THREADS=4 ./omp-demo0
Hello, world!
Hello, world!
Hello, world!
Hello, world!
```

“Hello, world” in OpenMP

```
/* omp-demo1.c */
#include <stdio.h>
#include <omp.h>

void say_hello( void )
{
    int my_rank = omp_get_thread_num();
    int thread_count = omp_get_num_threads();
    printf("Hello from thread %d of %d\n",
           my_rank, thread_count);
}

int main( void )
{
    #pragma omp parallel
    say_hello();

    return 0;
}
```

```
$ gcc -fopenmp omp-demo1.c -o omp-demo1
$ ./omp-demo1
Hello from thread 0 of 2
Hello from thread 1 of 2
$ OMP_NUM_THREADS=4 ./omp-demo1
Hello from thread 1 of 4
Hello from thread 2 of 4
Hello from thread 0 of 4
Hello from thread 3 of 4
```

```
/* omp-demo2.c */
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

void say_hello( void )
{
    int my_rank = omp_get_thread_num();
    int thread_count = omp_get_num_threads();
    printf("Hello from thread %d of %d\n",
           my_rank, thread_count);
}

int main( int argc, char* argv[] )
{
    int thr = atoi( argv[1] );
    #pragma omp parallel num_threads(thr)
    say_hello();

    return 0;
}
```

```
$ gcc -fopenmp omp-demo2.c -o omp-demo2
```

```
$ ./omp-demo2 2
```

```
Hello from thread 0 of 2
```

```
Hello from thread 1 of 2
```

```
$ ./omp-demo2 4
```

```
Hello from thread 1 of 4
```

```
Hello from thread 2 of 4
```

```
Hello from thread 0 of 4
```

```
Hello from thread 3 of 4
```

Setting the number of threads programmatically

```
/* omp-demo3.c */
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

void say_hello( void )
{
    int my_rank = omp_get_thread_num();
    int thread_count = omp_get_num_threads();
    printf("Hello from thread %d of %d\n",
           my_rank, thread_count);
}

int main( int argc, char* argv[] )
{
    omp_set_num_threads(4);
    #pragma omp parallel
    say_hello();

    return 0;
}
```

```
$ gcc -fopenmp omp-demo3.c -o omp-demo3
$ OMP_NUM_THREADS=8 ./omp-demo3
Hello from thread 1 of 4
Hello from thread 2 of 4
Hello from thread 0 of 4
Hello from thread 3 of 4
```

Warning

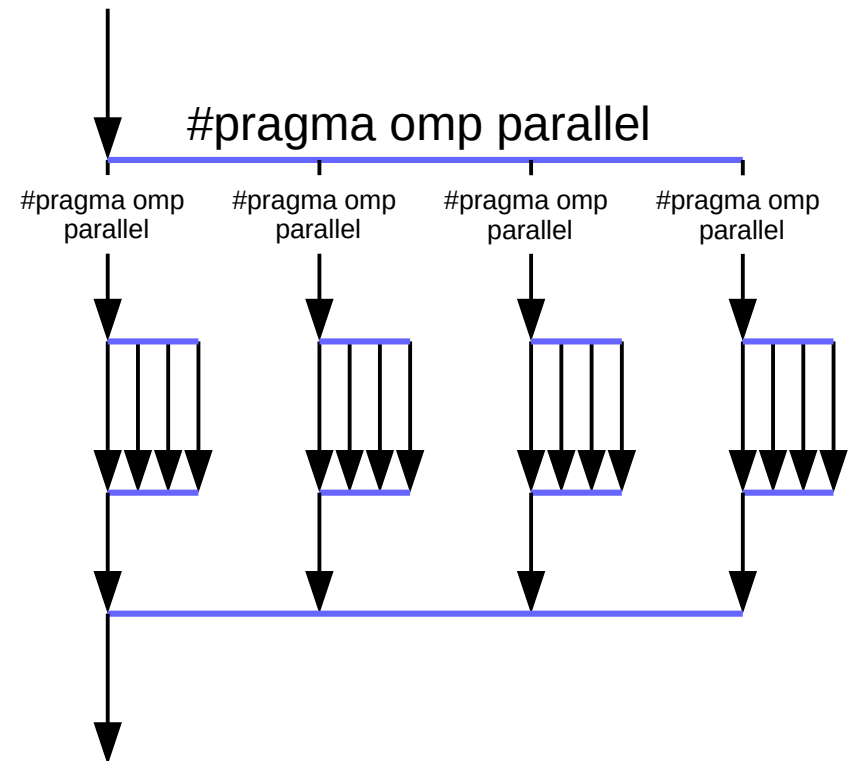
- `omp_get_num_threads()` returns the number of threads in the **currently active** team
 - If no team is active, the function returns 1
- `omp_get_max_threads()` returns the **maximum number of threads** that can be created

```
int main( int argc, char* argv[] )
{
    printf("Before par region: threads=%d, max=%d\n",
           omp_get_num_threads(), omp_get_max_threads());
    #pragma omp parallel
    {
        printf("Inside par region: threads=%d, max=%d\n",
               omp_get_num_threads(), omp_get_max_threads());
    }
    return 0;
}
```

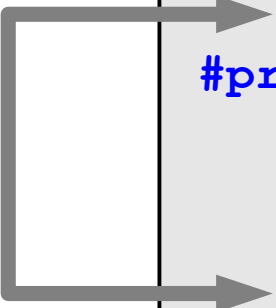
See [omp-demo4.c](#)

Nested parallelism

- In short: **DON'T**
- Not enabled by default; to enable, set the environment variable **OMP_NESTED=true**
 - **omp_get_num_threads()** returns the size of the innermost team is part of
 - **omp_get_thread_num()** returns the ID of the calling thread in the innermost team it is part of
- Nested parallelism is rarely useful
 - Might cause **oversubscription**, i.e., more threads than processors
- See **omp-nested.c**



Taking times



```
double tstart, tstop;
tstart = omp_get_wtime();
#pragma omp ...
{
    /* block of code to measure */
}
tstop = omp_get_wtime();
printf("Elapsed time: %f\n", tstop - tstart);
```

Remember: implicit barrier here

Scoping of variables

Scope

- In serial programming, the **scope** of a variable consists of the parts of a program where the variable can be used
- In OpenMP, the scope of a variable refers to the set of threads that can access the variable
- By default:
 - All variables that are visible **at the beginning** of a parallel block are **shared** across threads
 - All variables defined **inside** a parallel block are **private** to each thread

Scope of variables OpenMP

- **shared(x)** 
 - all threads access the same memory location
- **private(x)**
 - each thread has its own private copy of x
 - all local instances of x **are not initialized**
 - local updates to x are **lost** when exiting the parallel region
 - (OpenMP ≥ 3.0 only) the original value of x is retained at the end of the block
- **firstprivate(x)**
 - each thread has its own private copy of x
 - all copies of x **are initialized with the current value of x**
 - local updates to x are **lost** when exiting the parallel region
 - (OpenMP ≥ 3.0 only) the original value of x is retained at the end of the block
- **default(shared)** or **default(none)**
 - affects all the variables not specified in other clauses
 - **default(none)** ensures that you **must** specify the scope of each variable used in the parallel block that the compiler can not figure out by itself (**highly recommended!!**)

What is the output of this program?

```
/* omp-scope.c */
#include <stdio.h>

int main( void )
{
    int a=1, b=1, c=1, d=1;
    #pragma omp parallel num_threads(10) \
        private(a) shared(b) firstprivate(c)
    {
        printf("Hello World!\n");
        a++;
        b++;
        c++;
        d++;
    }
    printf("a=%d\n", a);
    printf("b=%d\n", b);
    printf("c=%d\n", c);
    printf("d=%d\n", d);
    return 0;
}
```

Hint: compile with -Wall

Sharing arrays

```
int a[3] = {10, 9, 8};  
int *b = (int*)malloc(5 * sizeof(*b) );
```

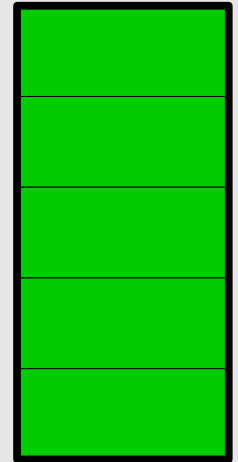
a[]

10	9	8
----	---	---

*b

0x0fba

0x0fba



```
#pragma omp parallel num_threads(3) firstprivate(a, b)  
{
```

```
}
```

Sharing arrays

```
int a[3] = {10, 9, 8};  
int *b = (int*)malloc(5 * sizeof(*b) );
```

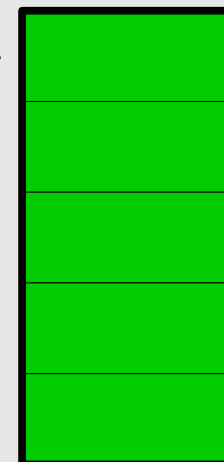
a[]

10	9	8
----	---	---

*b

0x0fba

0x0fba



```
#pragma omp parallel num_threads(3) firstprivate(a, b)  
{
```

Thread 0

a₀[]

10	9	8
----	---	---

*b₀

0x0fba

Thread 1

a₁[]

10	9	8
----	---	---

*b₁

0x0fba

Thread 2

a₂[]

10	9	8
----	---	---

*b₂

0x0fba

}

OpenMP Memory Model

OpenMP memory model (from OpenMP spec.)

- *OpenMP provides a relaxed-consistency, shared-memory model.*
- *All OpenMP threads have access to a place to store and to retrieve variables, called the memory. In addition, each thread is allowed to have its own **temporary view** of the memory.*
- *The temporary view can represent any kind of intervening structure, such as machine registers, cache, or other local storage.*
- *The temporary view of memory allows the thread to cache variables and thereby to avoid going to memory for every reference to a variable.*

OpenMP memory model (from OpenMP spec.)

- *The memory model has **relaxed-consistency** because a thread's temporary view of memory is not required to be consistent with memory at all times.*
- *A value written to a variable can remain in the thread's temporary view until it is forced to memory at a later time. Likewise, a read from a variable may retrieve the value from the thread's temporary view, unless it is forced to read from memory.*
- *OpenMP **flush operations** are used to enforce consistency between a thread's temporary view of memory and memory, or between multiple threads' view of memory.*

Flush operations

- `#pragma omp flush(var1, var2, ...)`
 - `#pragma omp flush` flushes all shared variables
- An implied flush directive appears in conjunction with the following directives:
 - `omp barrier`
 - Entry to and exit from `omp critical`.
 - Exit from `omp parallel`.
 - Exit from `omp for`.
 - Exit from `omp sections`.
 - Exit from `omp single`.

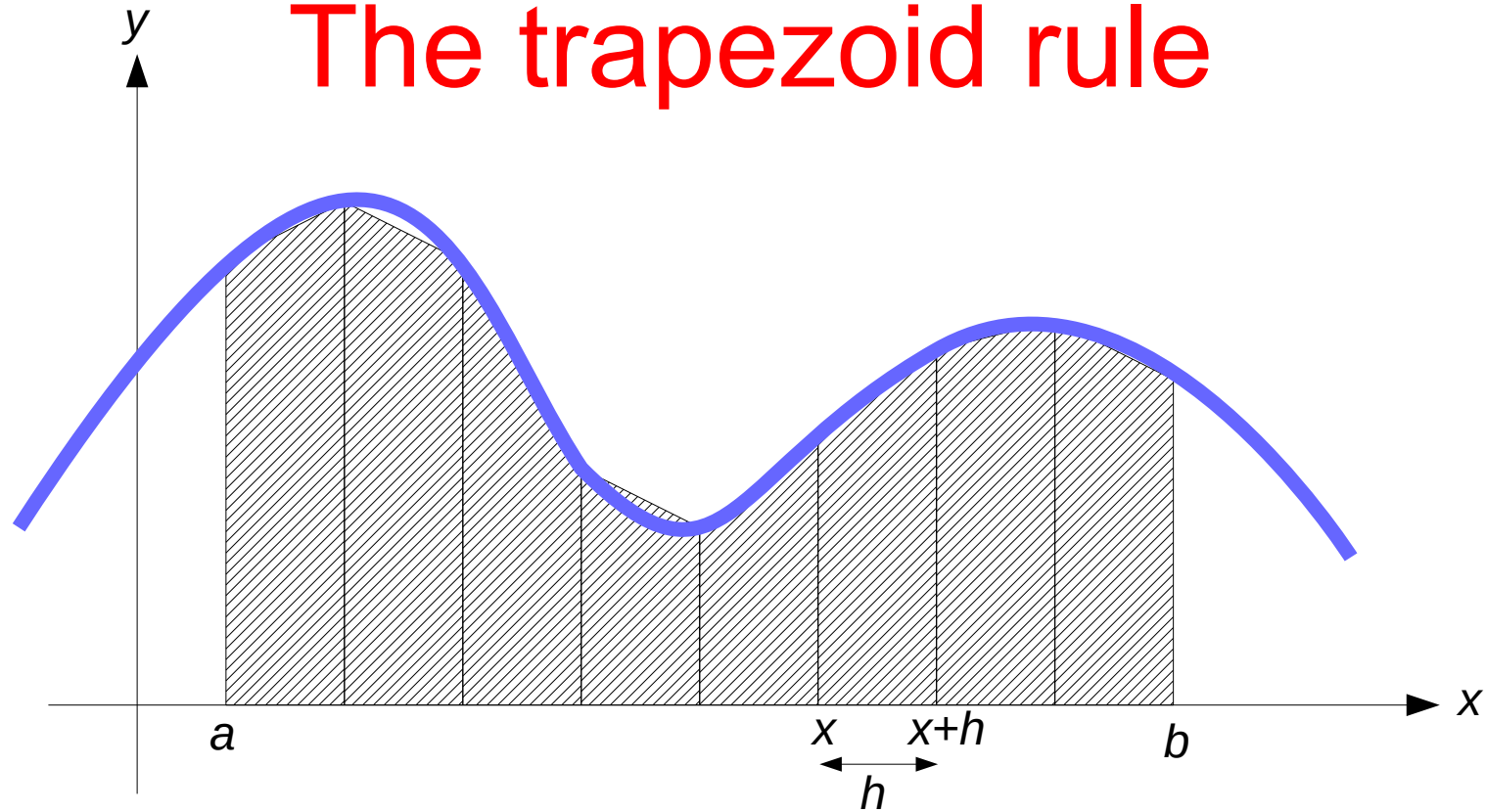
Example

Note: “benign”
race condition

```
int flag = 0;
#pragma omp parallel
{
    while (!flag) {
        /* ...lot of code here... */
        if (some_condition) {
            flag = 1;
        }
    }
}
```

Example: the trapezoid rule

The trapezoid rule



```
/* Serial trapezoid rule */  
h = (b-a)/n;  
result = 0;  
x = a;  
for (int i=0; i<n; i++) {  
    result += h*(f(x) + f(x+h))/2;  
    x += h;  
}  
return result;
```

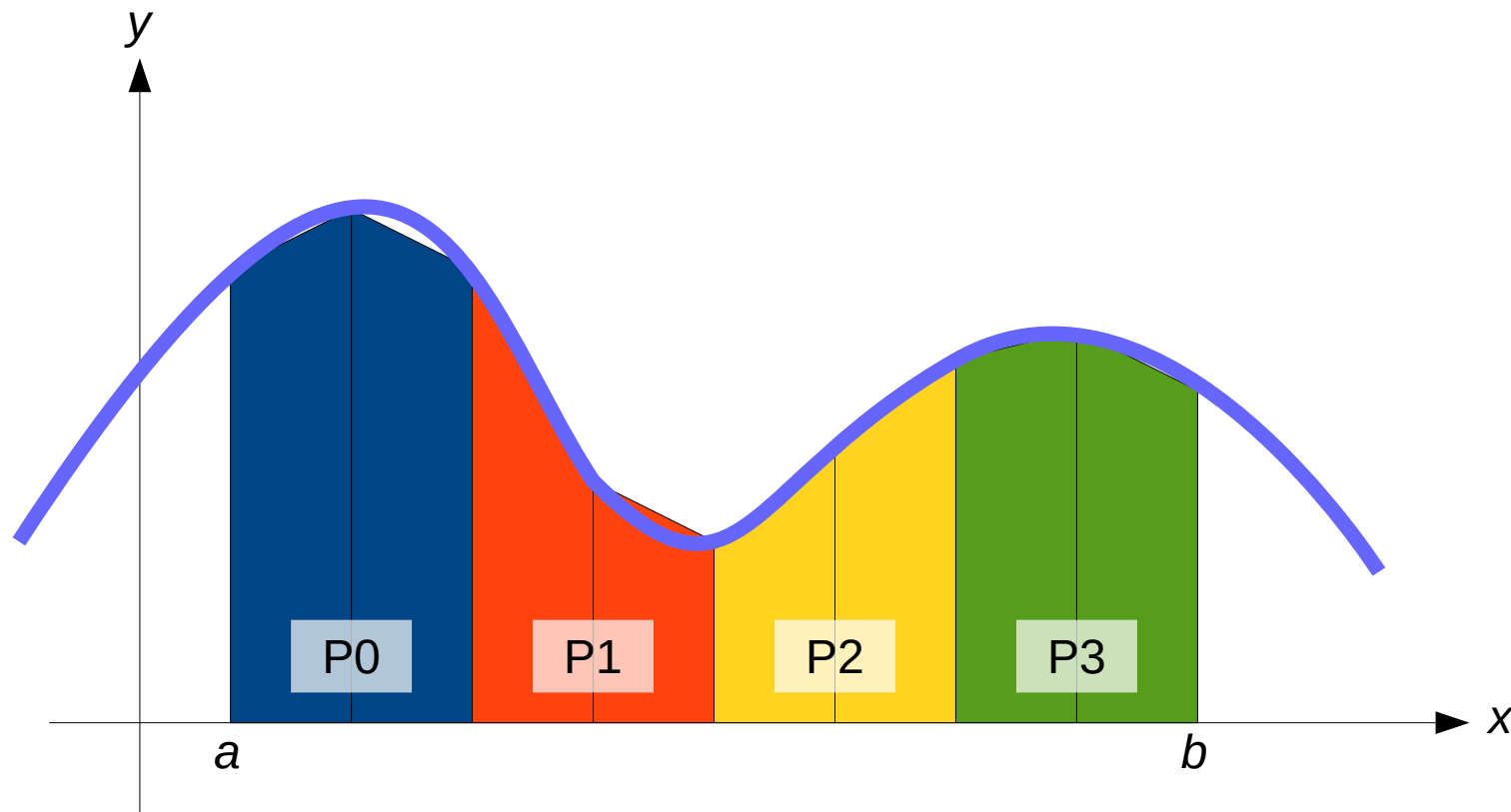
See [omp-trap.c](#)

A first OpenMP version

- Two types of tasks
 - computation of the areas of individual trapezoids
 - adding the areas of trapezoids
- Areas can be computed independently
- We assume that there are more trapezoids than OpenMP threads ($n \gg P$)

```
/* Serial trapezoid rule */  
h = (b-a)/n;  
result = 0;  
x = a;  
for (int i=0; i<n; i++) {  
    result += h*(f(a) + f(a+h))/2;  
    x += h;  
}  
return result;
```

Assigning trapezoids to threads



A first OpenMP version

- Split the n intervals across OpenMP threads
- Thread t stores its result in `partial_result[t]`
- The master sums all partial results
- See `trap0()`

A second OpenMP version

- Split the n intervals across OpenMP threads
- Thread t ...
 - ...stores its result in a **local** variable `partial_result`
 - ...updates the global result
- **omp atomic**
 - Protects updates to a *shared variable*
 - Updates must be of the form "read-update-write", e.g., `var += x`
- **omp critical**
 - Protects access to a *critical section*, which may consist of arbitrary instructions
 - All threads will eventually execute the critical section; however, only one thread at a time can be inside the critical block
- See `trap1()`

The **atomic** directive

- The **omp atomic** directive ensures that only one thread at the time updates a shared variable

```
#pragma omp parallel
{
    partial_result = ...
    #pragma omp atomic
    result += partial_result;
}
```

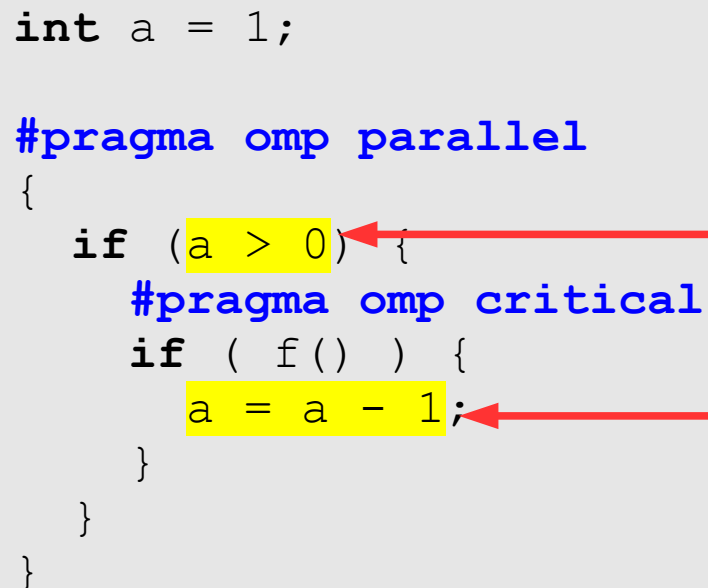
- The code above forces all threads to serialize during the update of the shared variable
 - This is not a real problem, since each thread will update the shared variable exactly once
 - A better solution is to use the **reduction** clause

Caution

- **critical** and **atomic** do **NOT** protect against race conditions caused by accesses to the shared variable(s) outside the critical/atomic blocks.
- Example: this **does not** work as expected

```
int a = 1;

#pragma omp parallel
{
    if (a > 0) {
        #pragma omp critical
        if ( f() ) {
            a = a - 1;
        }
    }
}
```



The reduction clause

- `reduction( <op> : <variable> )`



can be one of +, -, *, |, ^, &, |, &&, || (subtraction is handled in a slightly different way to ensure that the result is what is expected; note that **subtraction is deprecated** in recent OpenMP specifications)

```
#pragma omp parallel reduction(+:result)
{
    double partial_result = ...
    result += partial_result;
}
```

See [trap2\(\)](#)

Reduction operators

- A **reduction operator** is a **binary associative** operator such as addition or multiplication
 - An operator $\diamond$ is associative if $(a \diamond b) \diamond c = a \diamond (b \diamond c)$
- A **reduction** is a computation that repeatedly applies the same reduction operator to a sequence of operands to get a single result
 - $\diamond$ -reduce($x_0, x_1, \dots x_{n-1}$) = $x_0 \diamond x_1 \diamond \dots \diamond x_{n-1}$

How the **reduction** clause works

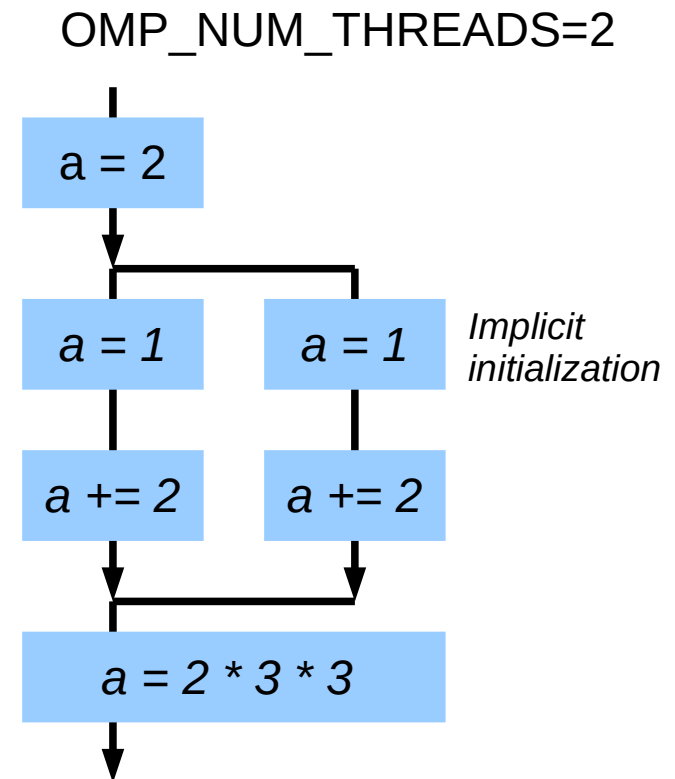
- One **private** copy of the reduction variable is created for each thread
- Each private copy is initialized with the neutral element of the reduction operator (e.g., 1 for *****, 0 for **+**)
- Each thread executes the parallel region
- When all threads finish, the reduction operator is applied to the *last* value of each local reduction variable, *and* the value the reduction variable had *before* the parallel region

```
/* omp-reduction.c */
#include <stdio.h>
int main( void )
{
    int a = 2;
    #pragma omp parallel reduction(*:a)
    {
        a += 2;
    }
    printf("%d\n", a);
    return 0;
}
```

```
$ OMP_NUM_THREADS=1 ./omp-reduction
6
$ OMP_NUM_THREADS=2 ./omp-reduction
18
$ OMP_NUM_THREADS=4 ./omp-reduction
162
```

How the **reduction** clause works

```
/* omp-reduction.c */
#include <stdio.h>
int main( void )
{
    int a = 2;
    #pragma omp parallel reduction(*:a)
    {
        /* implicit initialization a = 1 */
        a += 2;
    }
    printf("%d\n", a);
    return 0;
}
```



Some valid reduction operators

*OpenMP 3.1
and later*

Operator	Initial value
+	0
*	1
-	0
min	largest positive number
max	most negative number
&	~0
	0
^	0
&&	0
	1

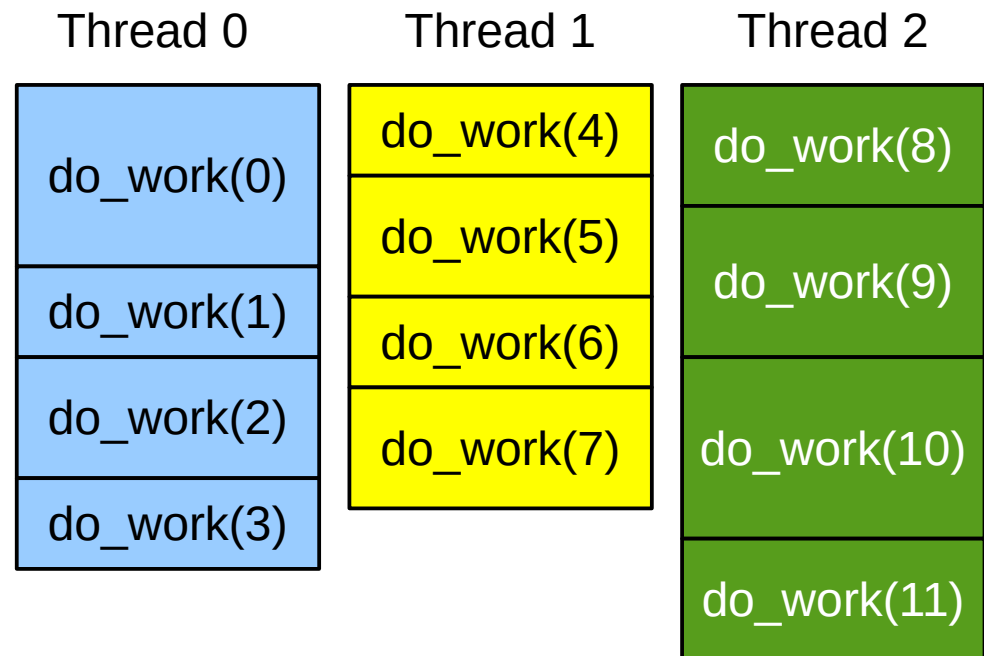
The **omp for** directive

- The **omp for** directive is used inside a parallel block
- Loop iterations are assigned to the threads of the current team (the ones created with **omp parallel**)
 - The loop variable is made **private** by default

```
int i;  
  
#pragma omp parallel  
{  
  #pragma omp for  
    for ( i=0; i<n; i++ ) {  
        do_work(i);  
    }  
}
```

Implied barriers here

Do not declare “i” as private; this is implied by #pragma omp for



The **parallel** **for** directive

- The **parallel** and **for** directives can be collapsed in a single **parallel for**

```
double trap3( double a, double b, int n )
{
    double result = 0;
    const double h = (b-a)/n;
    #pragma omp parallel for reduction(+:result)
    for ( int i = 0; i<n-1; i++ ) {
        result += h*(f(a+i*h) + f(a+(i+1)*h))/2;
    }
    return result;
}
```

See [trap3\(\)](#)

Legal forms for parallelizable *for* statements

```
for [
    index = start ;
    index < end
    index <= end
    index > end
    index >= end
    index++
    ++index
    index--
    --index
    index += incr
    index -= incr
    index = index + incr
    index = incr + index
    index = index - incr
]
```

- Variable `index` must have integer or pointer type (e.g., it can't be a float or double)
- The expressions `start`, `end`, and `incr` must have a compatible type. For example, if `index` is a pointer, `incr` must have integer type
- The expressions `start`, `end`, and `incr` must not change during execution of the loop
- Variable `index` can only be modified by the “increment expression” in the “`for`” statement

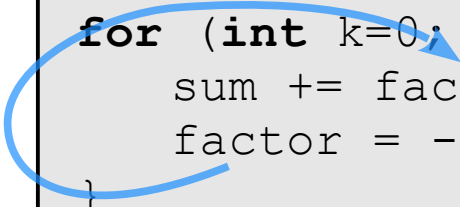
Data dependences

- It is incorrect to use a **parallel for** directive if data dependences are present
- Example: computation of PI

$$\pi = 4 \left\{ 1 - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} + \dots \right\} = 4 \sum_{k=0}^{+\infty} \frac{(-1)^k}{2k+1}$$

*Loop Carried
Dependence*

```
double factor = 1.0;
double sum = 0.0;
for (int k=0; k<n; k++) {
    sum += factor/(2*k + 1);
    factor = -factor;
}
pi_approx = 4.0 * sum;
```



Removing the data dependence

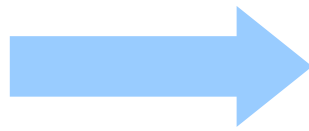
```
double factor;  
double sum = 0.0;  
#pragma omp parallel for private(factor) reduction(+:sum)  
for (int k=0; k<n; k++) {  
    if ( k % 2 == 0 ) {  
        factor = 1.0;  
    } else {  
        factor = -1.0;  
    }  
    sum += factor/(2*k + 1);  
}  
pi_approx = 4.0 * sum;
```

*factor must have
private scope*

Can my loop be parallelized?

- Proper solution: perform loop dependence analysis
 - Stay tuned...
- A quick-and-dirty test: run the loop *backwards*
 - If the program is still correct, the loop *might* be parallelizable
 - Not reliable, use with care

```
for (i=0; i<n; i++) {  
    /* loop body */  
}
```



```
for (i=n-1; i>=0; i--) {  
    /* loop body */  
}
```

Spot the bug

```
int i, j, n, m, temp;
...
#pragma omp parallel for private(temp)
for (i=0; i<n; i++){
    for (j=0; j<m; j++){
        temp = b[i]*c[j];
        a[i][j] = temp * temp + d[i];
    }
}
```

`schedule (type, chunksize)`

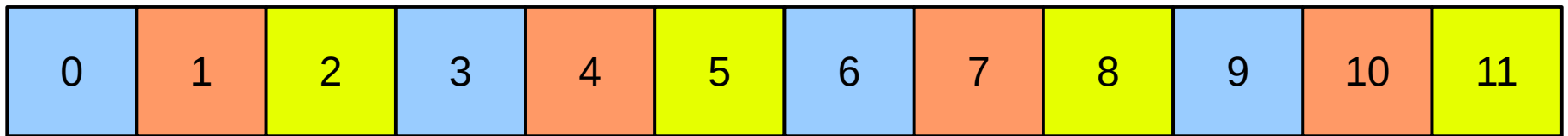
- **type** can be:
 - **static**: iterations are cyclically assigned to threads. If **chunksize** is not specified, then $chunksize = \lceil n_iterations / n_threads \rceil$
 - **dynamic** or **guided**: iterations are assigned to threads using a master-worker paradigm. Default **chunksize** is 1
 - **auto**: the compiler and/or the run-time system determines the schedule
 - **runtime**: the schedule is determined at run-time using the `OMP_SCHEDULE` environment variable, e.g.

```
OMP_SCHEDULE="static,16" ./my-prog.exe
```
- Default schedule is **implementation dependent**
 - GCC seems to use **static** by default

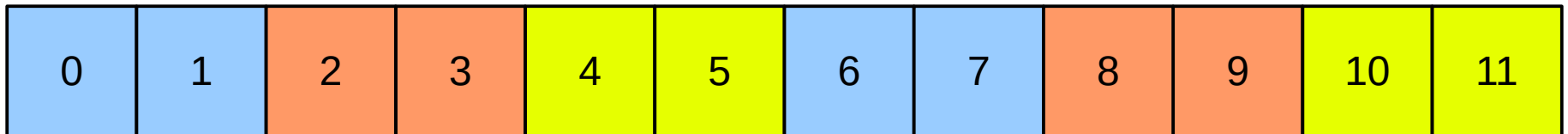
Example

- Twelve iterations 0, 1, ... 11 and three threads
- `schedule(static, 1)`

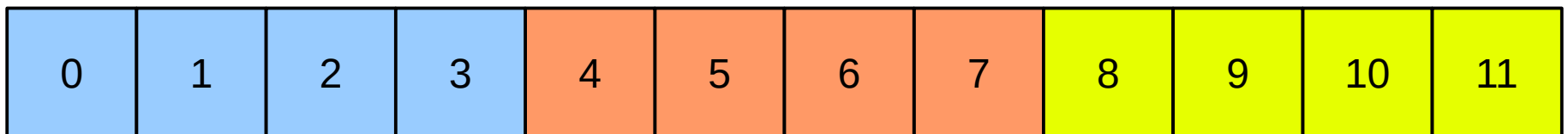
Thread 0
Thread 1
Thread 2



- `schedule(static, 2)`



- `schedule(static, 4)` ← Default chunksize in this case



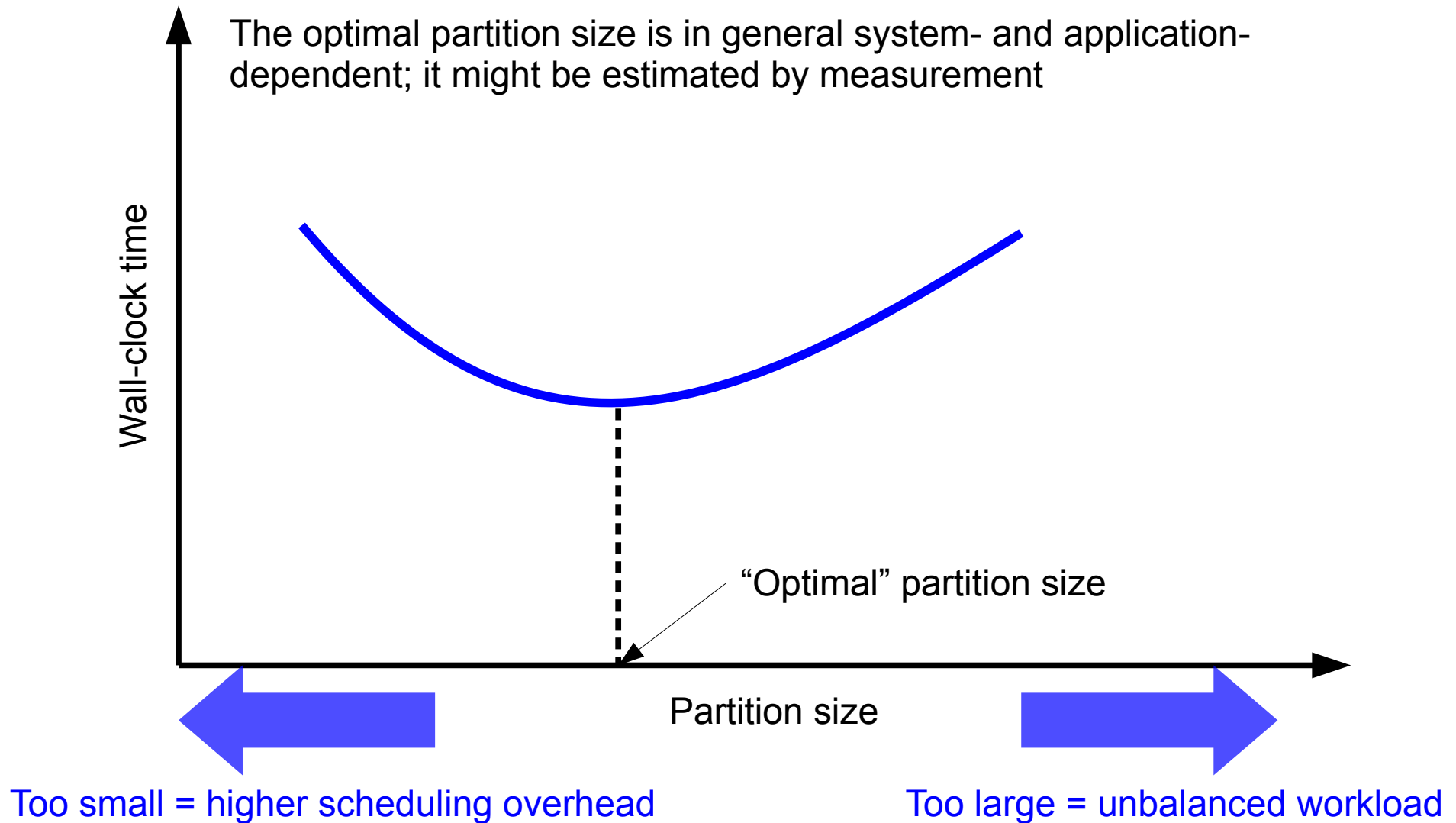
The Dynamic/Guided Schedule Types

- The iterations are broken up into chunks of **chunksize** consecutive iterations
 - However, in a **guided** schedule, as chunks are completed the size of the new chunks decreases
- Each thread executes a chunk, and at the end requests the next one from the run-time system
 - Master/Worker paradigm

Choosing a schedule clause

Clause	When to use	Note
static	Pre-determined and predictable work per iteration	Least work at runtime: scheduling done at compile-time
dynamic	Unpredictable, highly variable work per iteration	Most work at runtime: complex scheduling logic used at run-time

Choosing the partition size



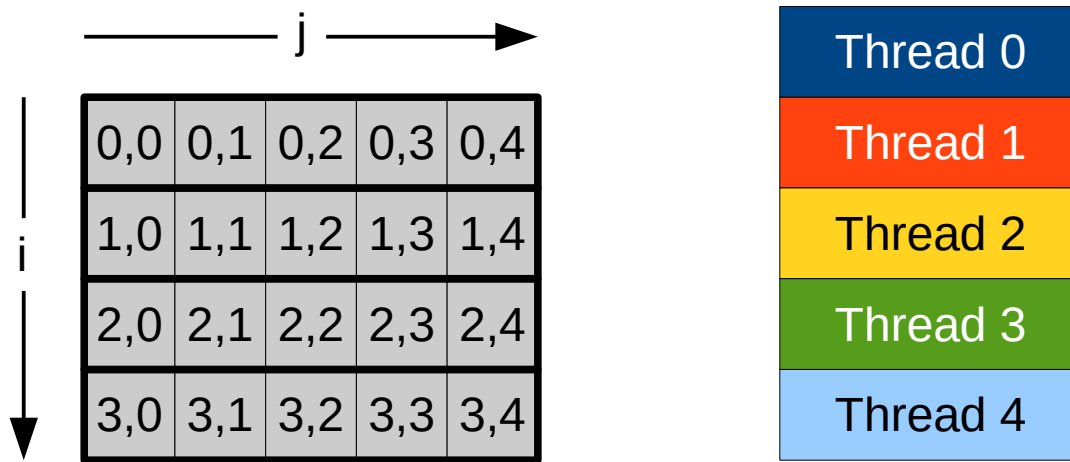
The collapse directive

- Specifies how many loops in a nested loop should be collapsed into one large iteration space, divided according to the schedule clause

```
int x, y;  
  
#pragma omp parallel for collapse(2)  
    for ( y = 0; y < ysize; y++ ) {  
        for ( x = 0; x < xsize; x++ ) {  
            drawpixel( x, y );  
        }  
    }
```

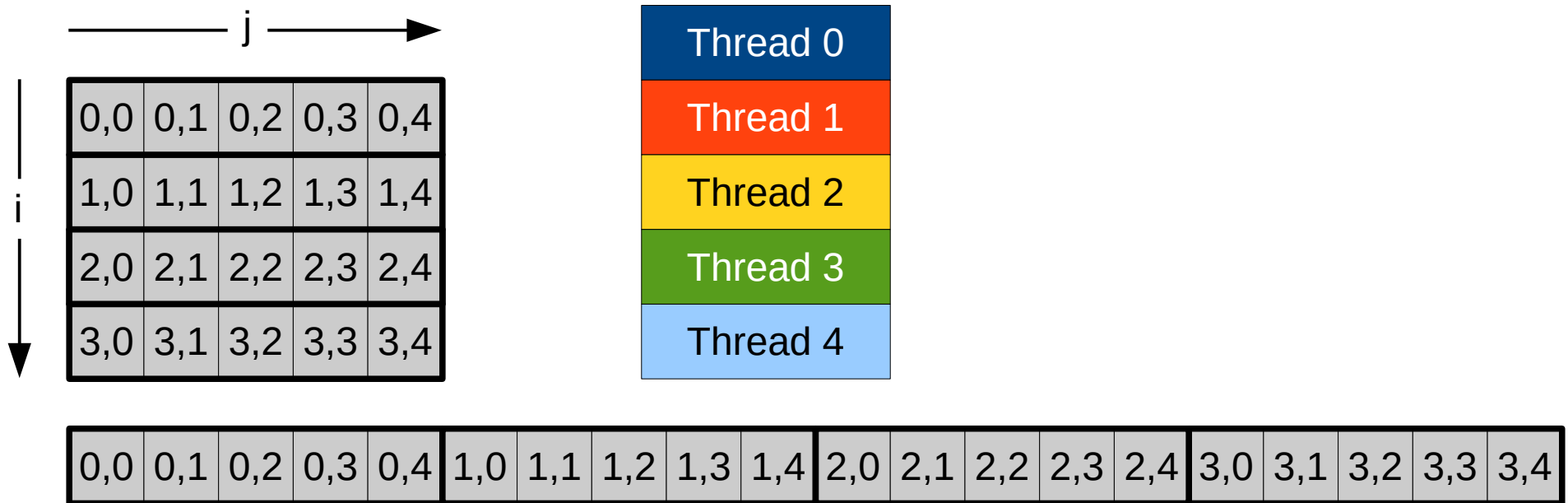
*collapse(2) makes
both loop variables
x and y **private***

How collapse works



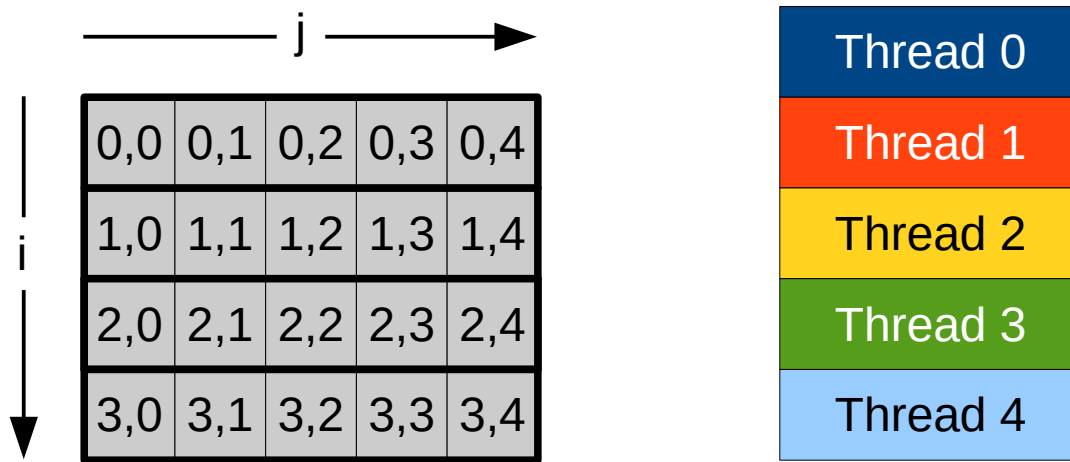
```
#pragma omp parallel for num_threads(5) collapse(2)
for (int i=0; i<4; i++) {
    for (int j=0; j<5; j++) {
        do_work(i,j);
    }
}
```

How collapse works

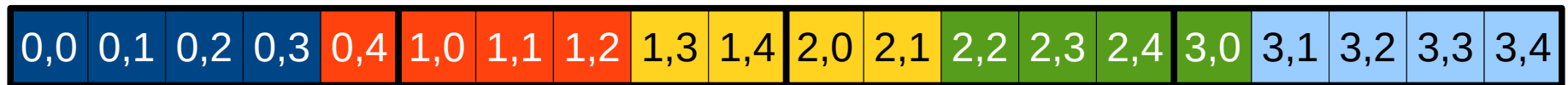


```
#pragma omp parallel for num_threads(5) collapse(2)
for (int i=0; i<4; i++) {
    for (int j=0; j<5; j++) {
        do_work(i,j);
    }
}
```

How collapse works

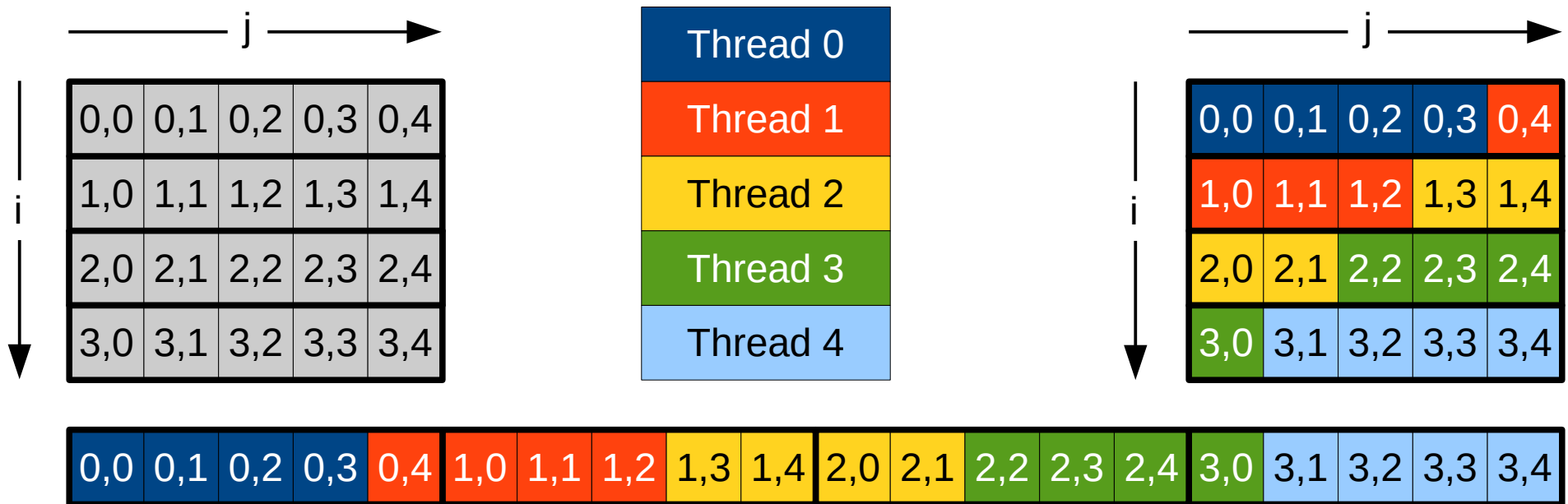


Note: the default assignment of iterations to threads is implementation-dependent (for gcc the default appears to be the **static** assignment).



```
#pragma omp parallel for num_threads(5) collapse(2)
for (int i=0; i<4; i++) {
    for (int j=0; j<5; j++) {
        do_work(i,j);
    }
}
```

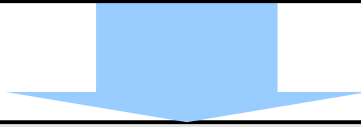
How collapse works



```
#pragma omp parallel for num_threads(5) collapse(2)
for (int i=0; i<4; i++) {
    for (int j=0; j<5; j++) {
        do_work(i,j);
    }
}
```


How collapse might be implemented

```
#pragma omp parallel for collapse(3)
for (int i=0; i<n; i++)
    for (int j=0; j<m; j++)
        for (int k=0; k<p; k++)
            foo(i, j, k);
```



```
#pragma omp parallel for
for (int idx=0; idx < n*m*p; idx++) {
    int tmp = idx;
    /* tmp = k + p * (j + i * m) */
    int k = tmp % p;
    tmp /= p;
    /* tmp = j + i * m */
    int j = tmp % m;
    tmp /= m;
    /* tmp = i */
    int i = tmp;
    foo(i, j, k);
}
```

In practice, much more complex:

- Lower/upper bounds can be arbitrary.
- Loops might iterate backwards.
- Handle <, <=, >, >=
- Non-unitary steps.
- ...

The collapse directive could introduce some overhead.

Mapping threads to CPUs

Mapping threads to CPUs

- For homogeneous multicore processors (even with Hyperthreading) the default is fine
 - Threads are spread across different physical cores first, the virtual CPUs are used
- The **OMP_PLACES** env. variable can be used on non-homogeneous processors (P-cores/E-cores) to specify where threads must be placed
- **OMP_PROC_BIND=true** prevents the OS scheduler to migrate threads to a different CPU
- You can use **htop** on the command line to see which CPUs are working

OMP_PLACES

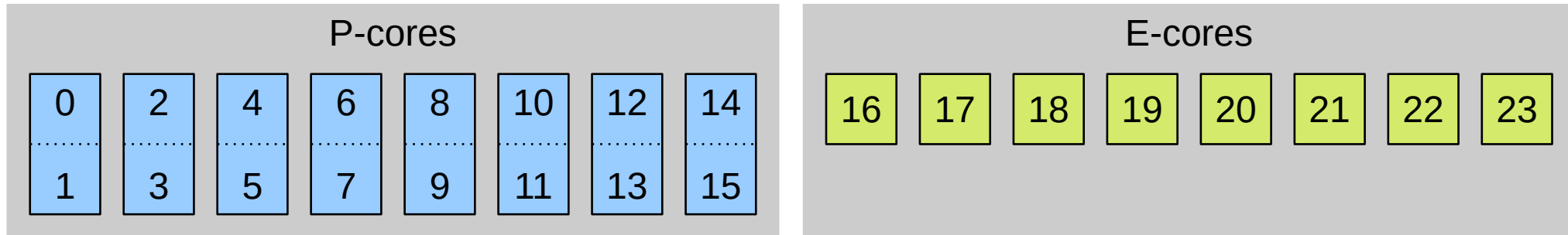
- **OMP_PLACES="0,1,2,3"**
 - Assign the first thread to CPU 0, the second to CPU 1, ...
 - Q: what if I launch more than 4 threads?
A: the list gets reused, i.e., the fifth thread is assigned to CPU 0 again
- **OMP_PLACES="0:4"**
 - Same as above, expands to {0}, {1}, {2}, {3}

OMP_PLACES

lower bound len stride
 ↓ ↓ ↓

- **OMP_PLACES="0:8:2"**
 - Read as: *"take 8 numbers, starting from 0, each one separated by 2 from the predecessor"*
 - Expands to {0}, {2}, {4}, {6}, {8}, {10}, {12}, {14}
- **OMP_DISPLAY_ENV=true** prints how **OMP_PLACES** is expanded, and also the values of other OpenMP environment variables

Example



- Suppose that we only want to use P-cores
- **`OMP_PLACES="0:8:2,1:8:2"`**
 - Expands to $\{0\}, \{2\}, \{4\}, \{6\}, \{8\}, \{10\}, \{12\}, \{14\},$
 $\{1\}, \{3\}, \{5\}, \{7\}, \{9\}, \{11\}, \{13\}, \{15\}$
 - Spreads threads to different cores first, and then fills “virtual” CPUs within the same core

```
$ OMP_NUM_THREADS=9 OMP_DISPLAY_ENV=true OMP_PLACES="0:8:2,1:8:2" ./omp-  
c-ray < sphfract.big.in > /dev/null
```

```
OPENMP DISPLAY ENVIRONMENT BEGIN
```

```
_OPENMP = '201511'
```

```
[host] OMP_DYNAMIC = 'FALSE'
```

```
[host] OMP_NESTED = 'FALSE'
```

```
[host] OMP_NUM_THREADS = '9'
```

```
[host] OMP_SCHEDULE = 'DYNAMIC'
```

```
[host] OMP_PROC_BIND = 'FALSE'
```

```
[host] OMP_PLACES = '{0},{2},{4},{6},{8},{10},{12},{14},{1},{3},{5},{7},{9},{11},{13},{15}'
```

```
[host] OMP_STACKSIZE = '0'
```

```
[host] OMP_WAIT_POLICY = 'PASSIVE'
```

```
[host] OMP_THREAD_LIMIT = '4294967295'
```

```
[host] OMP_MAX_ACTIVE_LEVELS = '1'
```

```
[host] OMP_NUM_TEAMS = '0'
```

```
[host] OMP_TEAMS_THREAD_LIMIT = '0'
```

```
[all] OMP_CANCELLATION = 'FALSE'
```

```
[all] OMP_DEFAULT_DEVICE = '0'
```

```
[all] OMP_MAX_TASK_PRIORITY = '0'
```

```
[all] OMP_DISPLAY_AFFINITY = 'FALSE'
```

```
[host] OMP_AFFINITY_FORMAT = 'level %L thread %i affinity %A'
```

```
[host] OMP_ALLOCATOR = 'omp_default_mem_alloc'
```

```
[all] OMP_TARGET_OFFLOAD = 'DEFAULT'
```

```
OPENMP DISPLAY ENVIRONMENT END
```

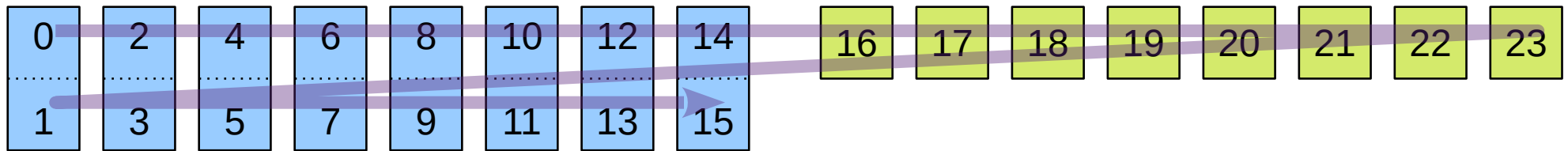
Example

- Transposed matrix-matrix multiply
- Matrix size 2048×2048

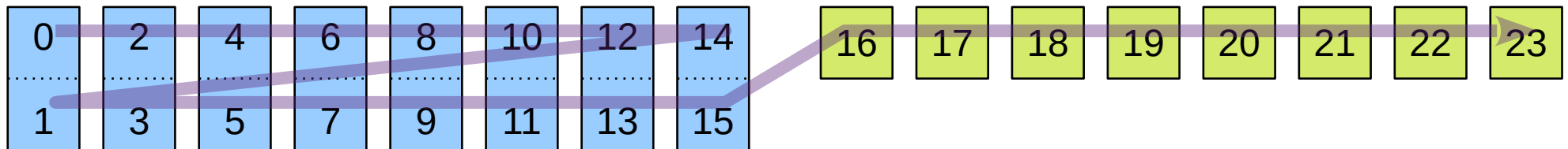
Processor	12th Gen Intel(R) Core(TM) i9-12900F
Clock freq	5.1 GHz
Cores	8 P-cores + 8 E-cores
Hyperthreading	yes (on P-cores only)
RAM	64 GB
OS	Ubuntu Linux 24.04
Compiler	gcc (Ubuntu 13.3.0-6ubuntu2~24.04) 13.3.0
Flags	-std=c99 -Wall -Wpedantic -fopenmp

Partition and mapping policies

- **static** schedule; default mapping
- **static** schedule; 8P, 8E, 8P mapping
 - `OMP_PLACES="0:8:2, 16:8:1, 1:8:2"`

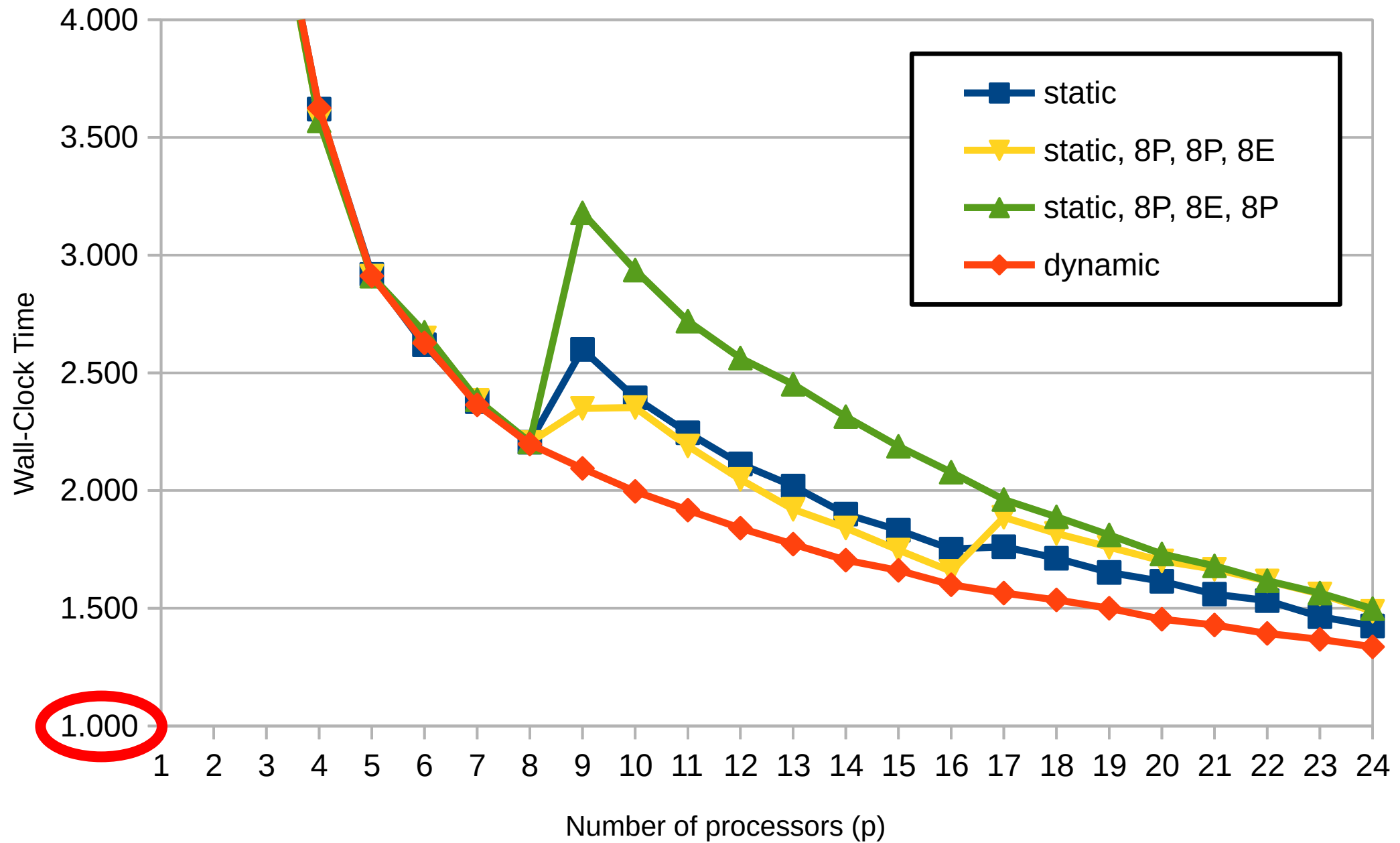


- **static** schedule; 8P, 8P, 8E mapping
 - `OMP_PLACES="0:8:2, 1:8:2, 16:8:1"`



- **dynamic** schedule; default mapping

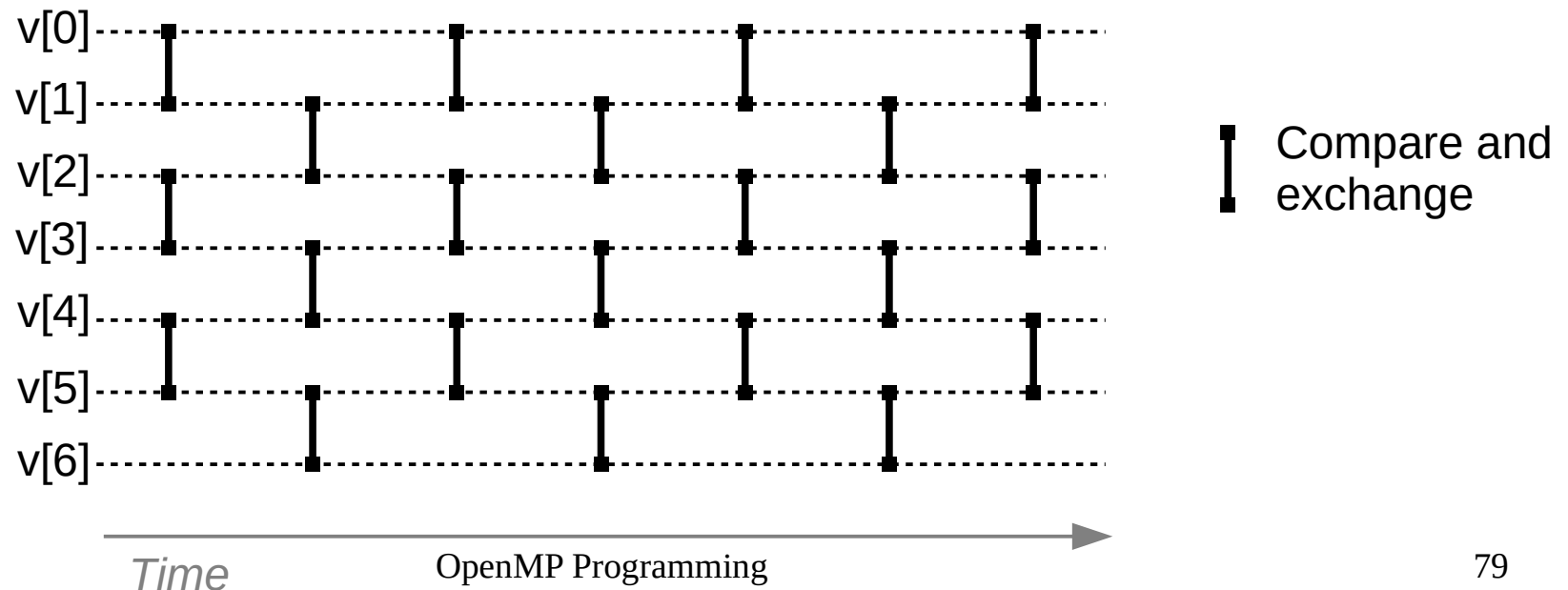
Wall-Clock Time



Example: Odd-Even Transposition Sort

Serial Odd-Even Transposition Sort

- Variant of bubble sort
- Compare all (even, odd) pairs of adjacent elements, and exchange them if in the wrong order
- Then compare all (odd, even) pairs, exchanging if necessary; repeat the step above



Serial Odd-Even Transposition Sort

- Variant of bubble sort
- Compare all (even, odd) pairs of adjacent elements, and exchange them if in the wrong order
- Then compare all (odd, even) pairs, exchanging if necessary; repeat the step above

```
for (int phase = 0; phase < n; phase++) {  
    if ( phase % 2 == 0 ) {  
        for (int i=0; i<n-1; i+=2) {  
            if (v[i] > v[i+1]) swap( &v[i], &v[i+1] );  
        }  
    } else {  
        for (int i=1; i<n-1; i+=2) {  
            if (v[i] > v[i+1]) swap( &v[i], &v[i+1] );  
        }  
    }  
}
```

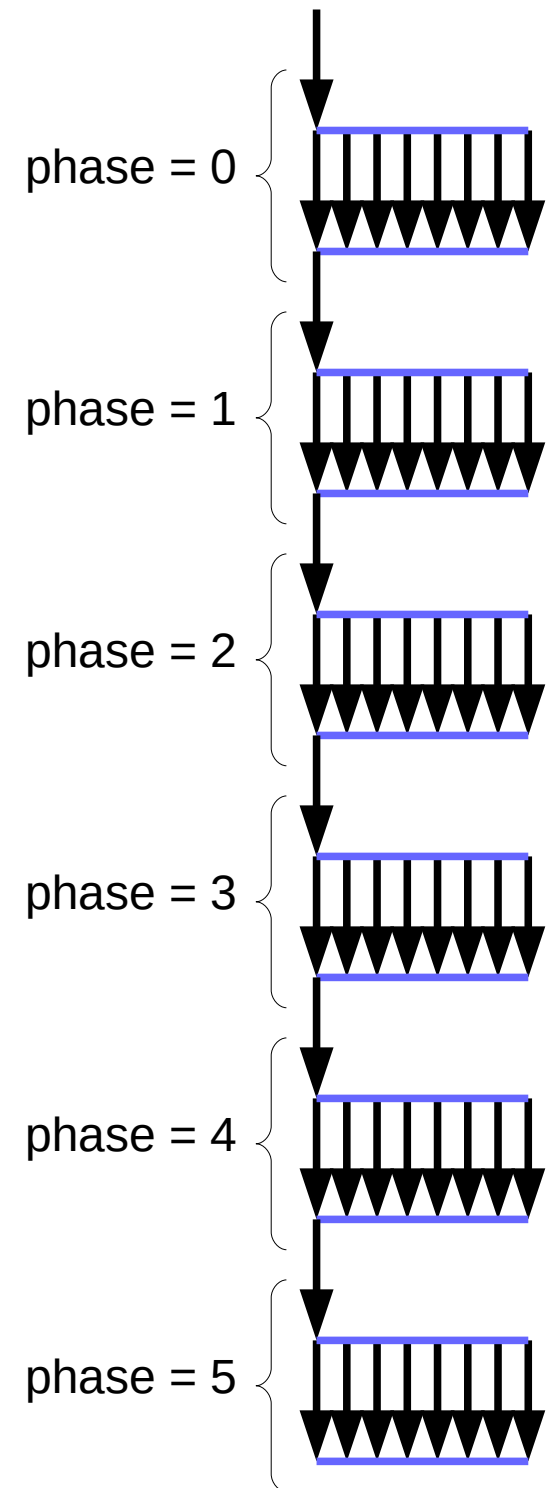
First OpenMP Odd-Even Sort

```
for (int phase = 0; phase < n; phase++) {  
    if ( phase % 2 == 0 ) {  
#pragma omp parallel for default(none) shared(v,n)  
        for (int i=0; i<n-1; i+=2) {  
            if (v[i] > v[i+1]) swap( &v[i], &v[i+1] );  
        }  
    } else {  
#pragma omp parallel for default(none) shared(v,n)  
        for (int i=1; i<n-1; i+=2) {  
            if (v[i] > v[i+1]) swap( &v[i], &v[i+1] );  
        }  
    }  
}
```

First OpenMP Odd-Even

```
for (int phase = 0; phase < n; phase++) {  
    if ( phase % 2 == 0 ) {  
#pragma omp parallel for default(none) shared(v)  
        for (int i=0; i<n-1; i+=2) {  
            if (v[i] > v[i+1]) swap( &v[i], &v[i+1])  
        }  
    } else {  
#pragma omp parallel for default(none) shared(v)  
        for (int i=1; i<n-1; i+=2) {  
            if (v[i] > v[i+1]) swap( &v[i], &v[i+1])  
        }  
    }  
}
```

OpenMP Programming



First OpenMP Odd-Even Sort

- The team of threads is being created/destroyed at each **omp parallel for** region.
- This may introduce some overhead, depending on the OpenMP implementation.
- We can nest **omp for** inside **omp parallel** to recycle the threads from the same team.
- Notes
 - Threads **do not synchronize** at the beginning of the **omp for** region
 - As soon as a thread arrives, it starts a chunk of the iteration regardless of what other threads are doing
 - Threads **do synchronize** at the end of the **omp for**

Second OpenMP Odd-Even Sort

```
#pragma omp parallel default(none) shared(v,n)
for (int phase = 0; phase < n; phase++) {
    if ( phase % 2 == 0 ) {
        #pragma omp for
        for (int i=0; i<n-1; i+=2) {
            if (v[i] > v[i+1]) swap( &v[i], &v[i+1] );
        }
    } else {
        #pragma omp for
        for (int i=1; i<n-1; i+=2 ) {
            if (v[i] > v[i+1]) swap( &v[i], &v[i+1] );
        }
    }
}
```

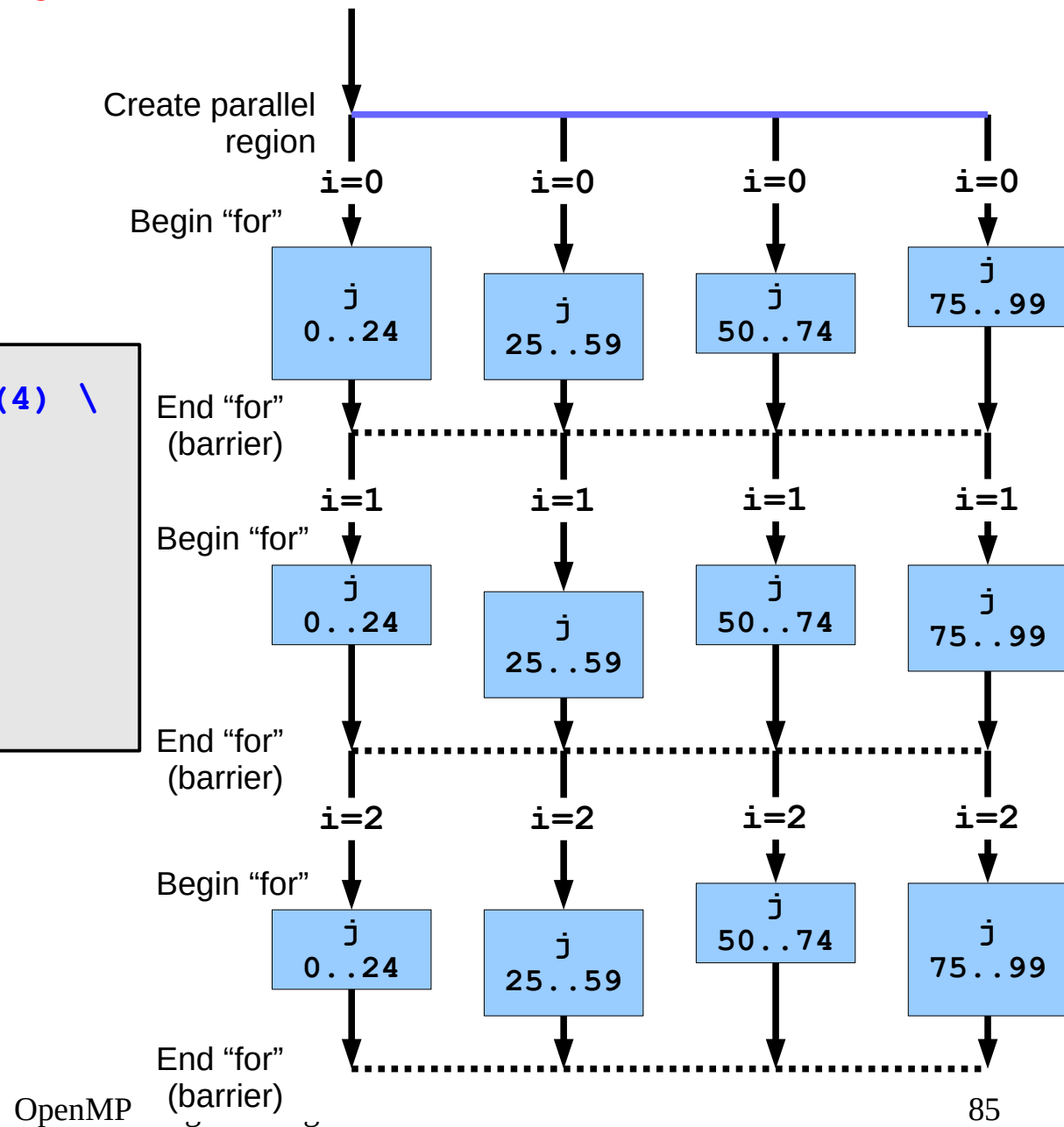
Create team

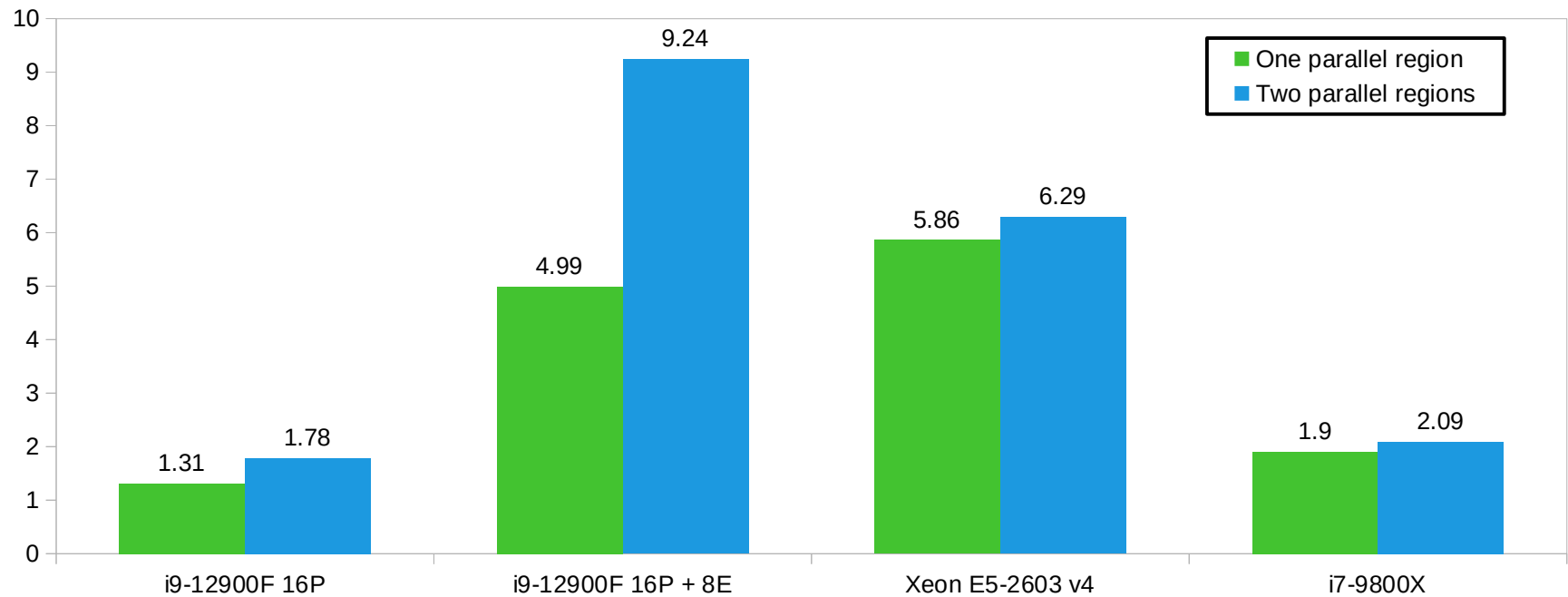
#pragma omp for uses the threads from the team created with #pragma omp parallel

See [omp-odd-even.c](#)

Why it works

```
#pragma omp parallel num_threads(4) \
    default(none)
for (int i = 0; i < 20; i++) {
    #pragma omp for
        for (int j=0; j < 100; j++) {
            do_work(i, j);
        }
}
```





	12th Gen Intel(R) Core(TM) i9-12900F Ubuntu 24.04.3 gcc 13.3.0 8+8 P-cores, 8 E-cores	Intel(R) Xeon(R) CPU E5-2603 v4 Ubuntu 20.04.6 gcc 9.4.0 12 cores	Intel(R) Core(TM) i7-9800X CPU gcc 13.3.0 8+8 cores
One parallel region	1.31 (using 16P) 4.99 (using 16P+8E)	5.86	1.90
Two parallel regions	1.78 (using 16P) 9.24 (using 16P+8E)	6.29	2.09

Array length=100000, 5 independent replications for each test.
omp-odd-even compiled with -std=c99 -Wall -Wpedantic -fopenmp

Synchronization directives

- **#pragma omp barrier**
 - All threads in the currently active team must reach this point before they are allowed to proceed
- **#pragma omp master**
 - Marks a parallel region which is executed by the master only (the thread with rank = 0); other threads just skip the region
 - There is **no barrier** at the end of the block
- **#pragma omp single**
 - Marks a parallel region which is executed only once, by the **first thread reaching it**, whichever it is; other threads just skip the region
 - **There is an implicit barrier** at the end of the block

Example

```
#pragma omp parallel
{
    foo();

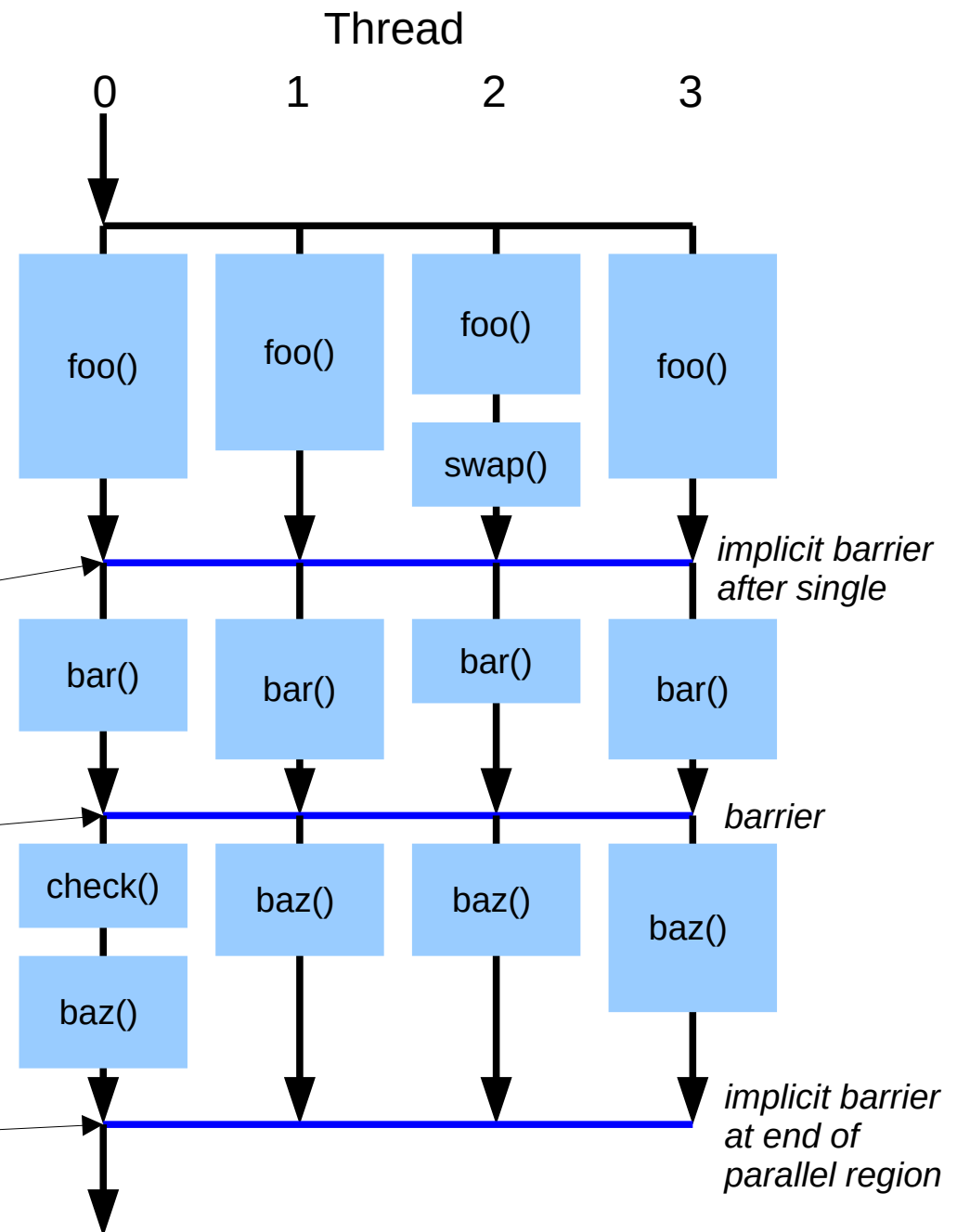
    #pragma omp single
    swap();

    bar();

    #pragma omp barrier

    #pragma omp master
    check();

    baz();
}
```



Work-sharing constructs

OpenMP tasking constructs

- Not all programs have simple loops that OpenMP can parallelize
- Example: linked list traversal
 - Each node of the linked list is "processed" independently from other nodes

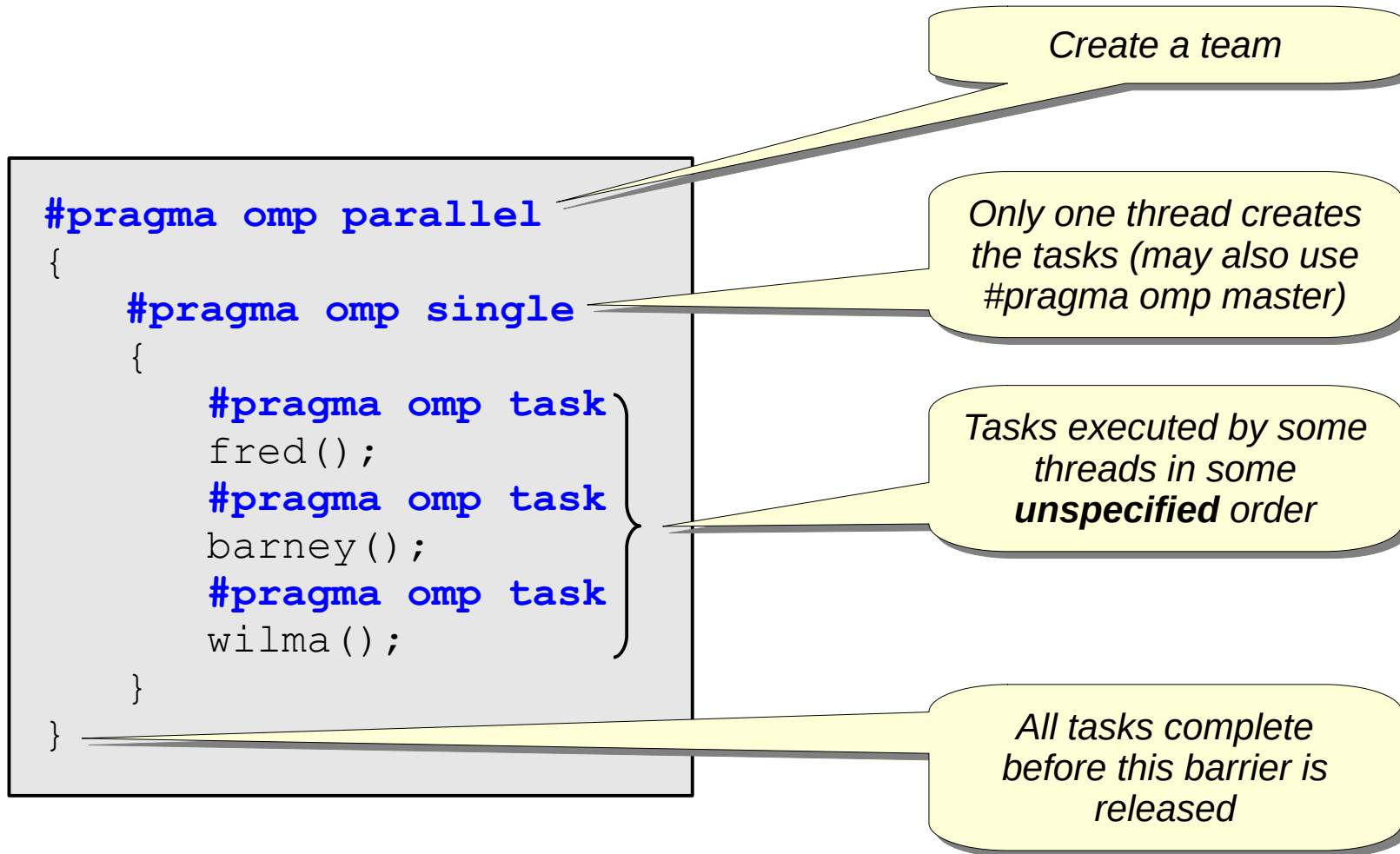
```
ListNode *p = head;
while (p) {
    process(p);
    p = p->next;
}
```

- OpenMP **parallel for** works only for loops where the iteration count can be known in advance at runtime

What are OpenMP tasks?

- Tasks are independent units of work
- Tasks are composed of:
 - code to execute
 - data to compute with
- Threads are assigned to perform the work of each task
 - The thread that encounters the task construct may execute the task immediately
 - The threads may defer execution until later
- Tasks can be nested: a task may generate other tasks

#pragma omp task



Data scoping with tasks

- Variables within a task construct are evaluated at the time a task is **executed**; this might happen long after a task is **created**
- You can specify the scope of variables within a **#pragma omp task** directive
- Variables can be **shared**, **private** or **firstprivate**
 - If a variable is **shared** on a task construct, the references to it inside the construct are to the storage with that name at the point where the task was encountered
 - If a variable is **private** on a task construct, the references to it inside the construct are to new uninitialized storage that is created when the task is executed
 - If a variable is **firstprivate** on a construct, the references to it inside the construct are to new storage that is created and initialized with the value of the existing storage of that name when the task is encountered

Data scoping with tasks

Scope of variable at task creation	Scope of variable inside the task
<code>shared</code>	<code>shared</code>
<code>private</code>	<code>firstprivate</code>
<code>firstprivate</code>	<code>firstprivate</code>

- The behavior you want for tasks is usually `firstprivate`, because the task may not be executed until later (and variables may have gone out of scope)
 - Variables that are `private` when the task construct is encountered are `firstprivate` by default

Example

```
int a = 1;

void foo( void )
{
    int b = 2, c = 3;
    #pragma omp parallel private(b)
    {
        int d = 4; // a,c shared; b private, d private
        #pragma omp task
        {
            int e = 5;
            // Scope of a: shared, value 1
            // Scope of b: firstprivate, uninitialized
            // Scope of c: shared, value 3
            // Scope of d: private => firstprivate, value 4
            // Scope of e: private, value 5
        }
    }
}
```

a=1

b=?

c=3

d=4

e=5

Example:

Linked list traversal with tasks

```
ListNode *p; // p shared
#pragma omp parallel
{
    #pragma omp single
    {
        p = head;
        while (p) {
            #pragma omp task
            process(p);
            p = p->next;
        }
    }
}
```

WRONG

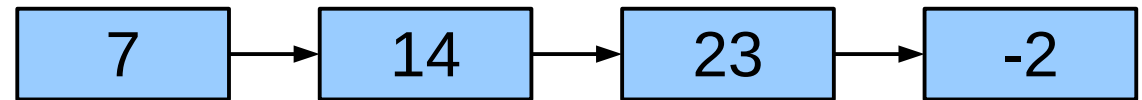
*according to the rules of
visibility in a task construct,
p is shared to each task*

Example:

Linked list traversal with tasks

```
ListNode *p;  
#pragma omp parallel  
{  
  #pragma omp single  
  {  
    p = head;  
    while (p) {  
      #pragma omp task  
      process(p);  
      p = p->next;  
    }  
  }  
}
```

WRONG



bag of tasks

process(**p**);

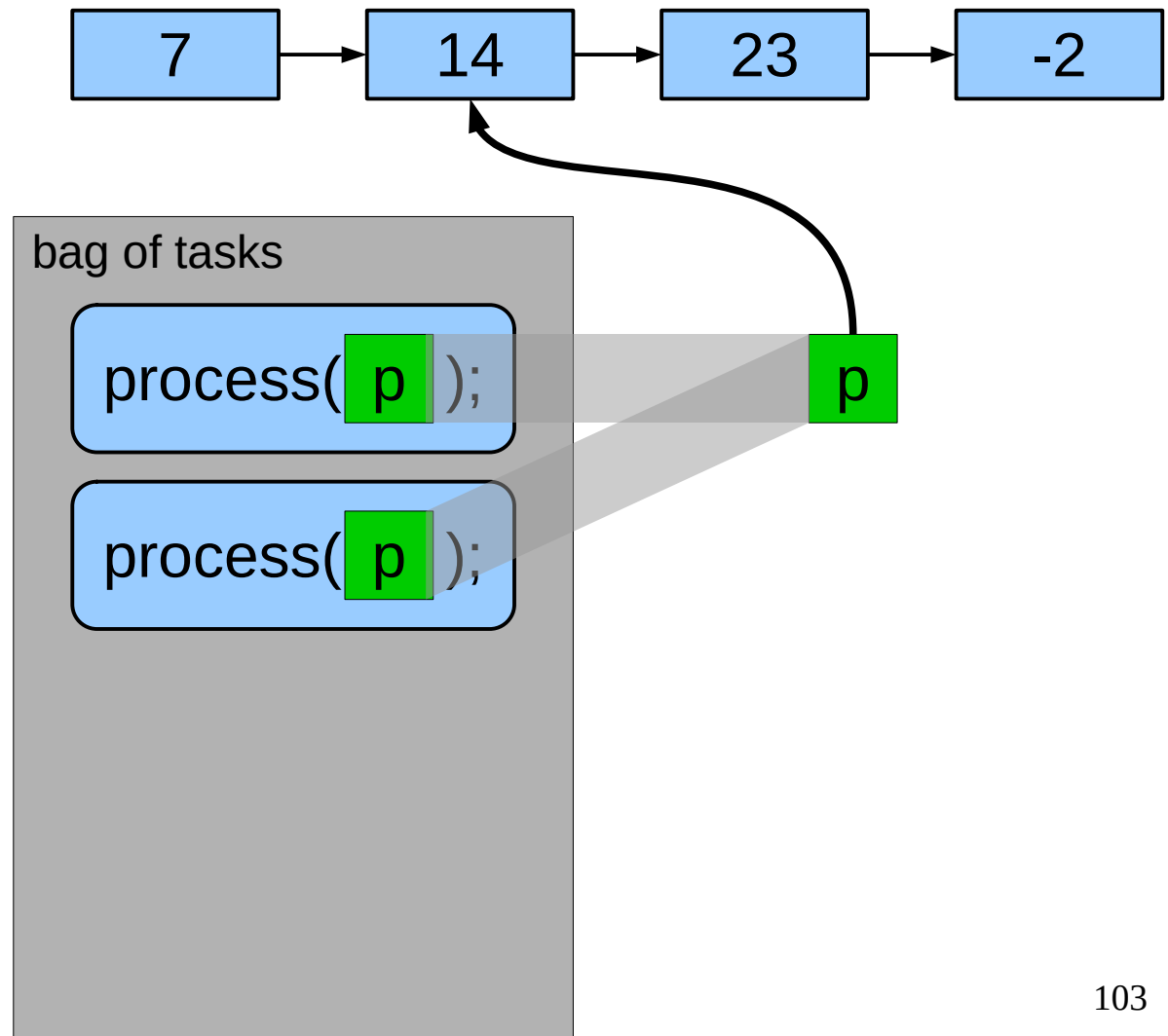
p

Example:

Linked list traversal with tasks

```
ListNode *p;  
#pragma omp parallel  
{  
  #pragma omp single  
  {  
    p = head;  
    while (p) {  
      #pragma omp task  
      process(p);  
      p = p->next;  
    }  
  }  
}
```

WRONG

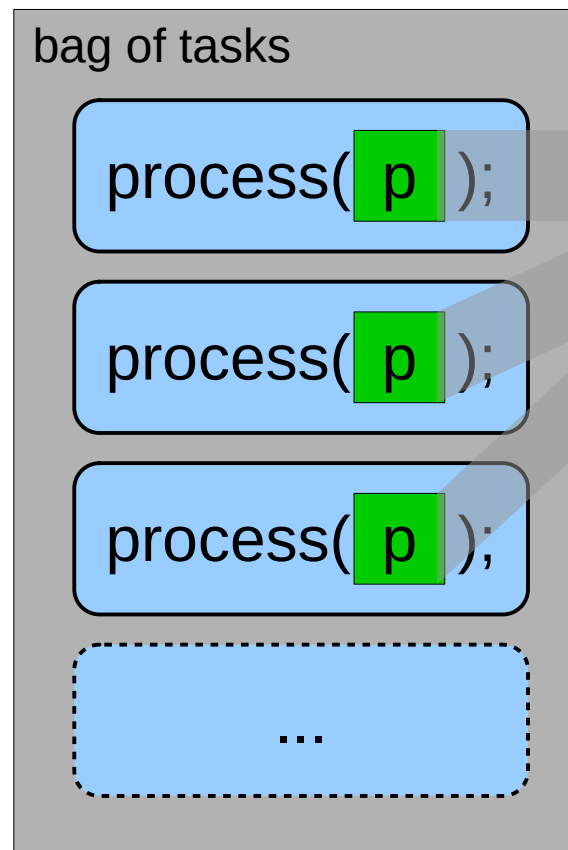
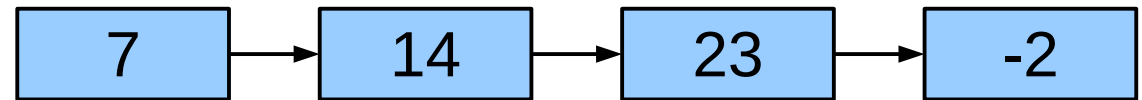


Example:

Linked list traversal with tasks

```
ListNode *p;  
#pragma omp parallel  
{  
  #pragma omp single  
  {  
    p = head;  
    while (p) {  
      #pragma omp task  
      process(p);  
      p = p->next;  
    }  
  }  
}
```

WRONG



Each task shares the same value of *p*, which might be different when the task is actually executed

Example:

Linked list traversal with tasks

```
#pragma omp parallel
{
  #pragma omp single
  {
    ListNode *p = head; // p private
    while (p) {
      #pragma omp task
      process(p);
      p = p->next;
    }
  }
}
```

CORRECT

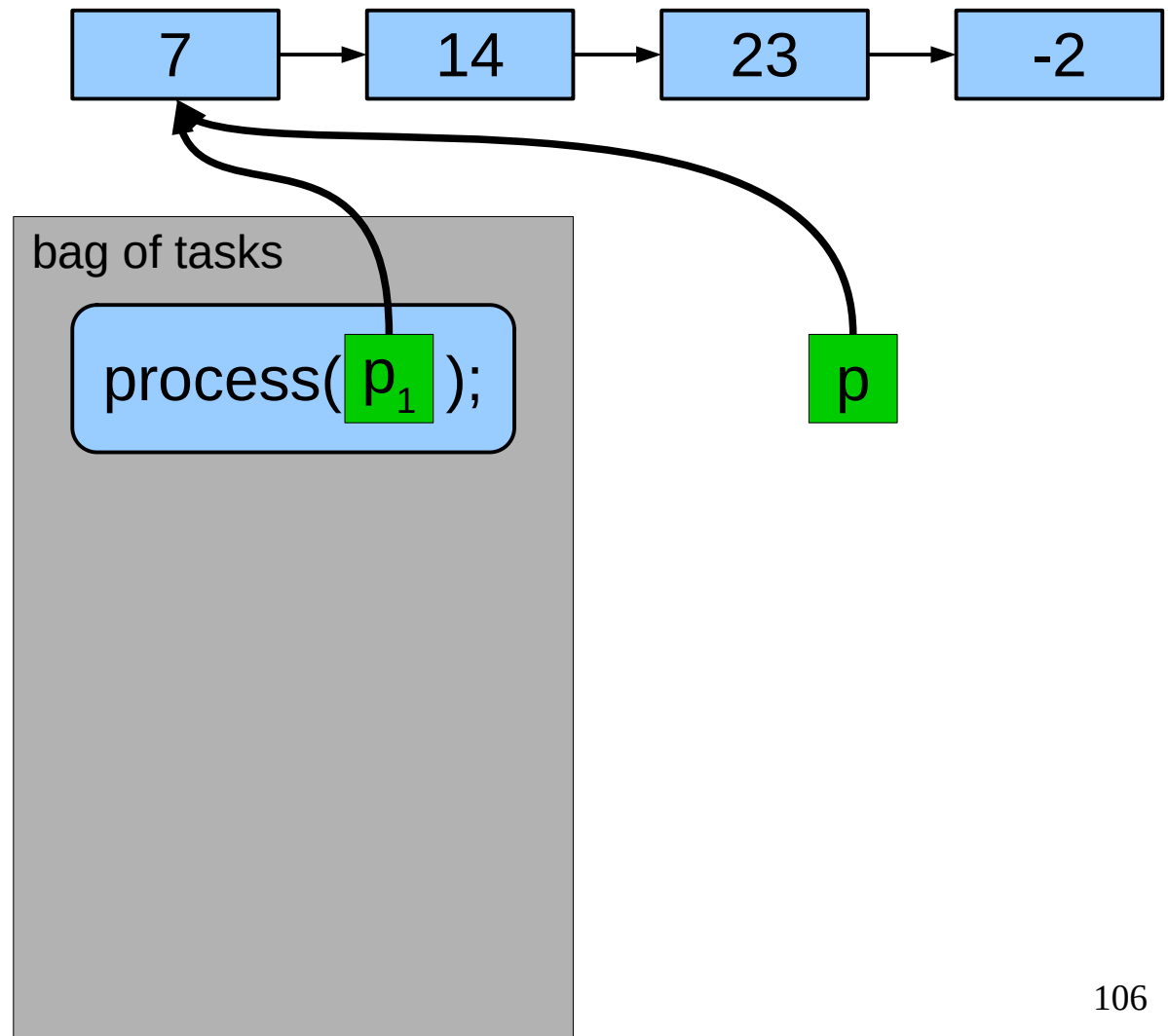
according to the rules of visibility in a task construct, p is firstprivate to each task

Example:

Linked list traversal with tasks

```
#pragma omp parallel
{
  #pragma omp single
  {
    ListNode *p = head;
    while (p) {
      #pragma omp task
      process(p);
      p = p->next;
    }
  }
}
```

CORRECT

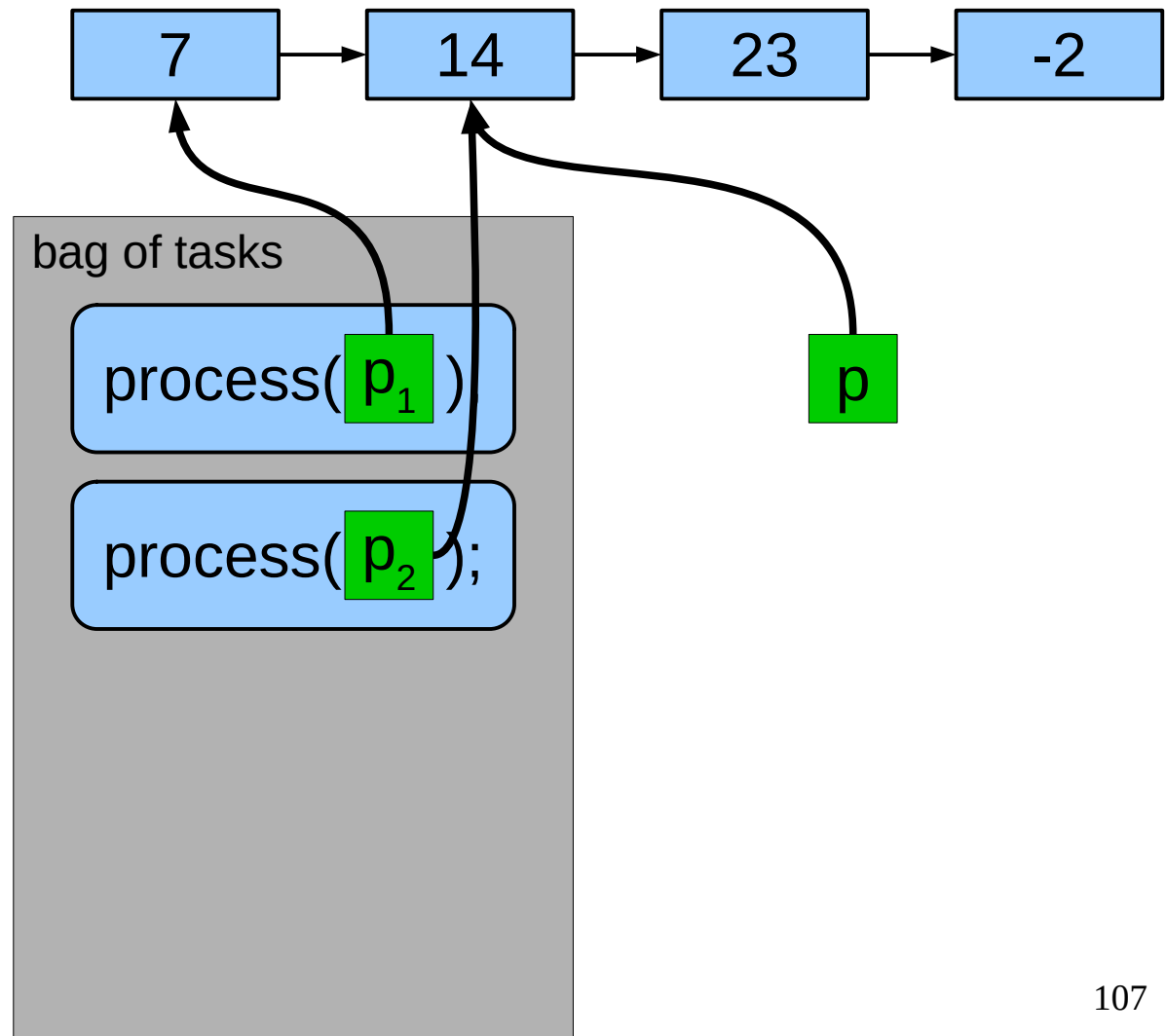


Example:

Linked list traversal with tasks

```
#pragma omp parallel
{
  #pragma omp single
  {
    ListNode *p = head;
    while (p) {
      #pragma omp task
      process(p);
      p = p->next;
    }
  }
}
```

CORRECT

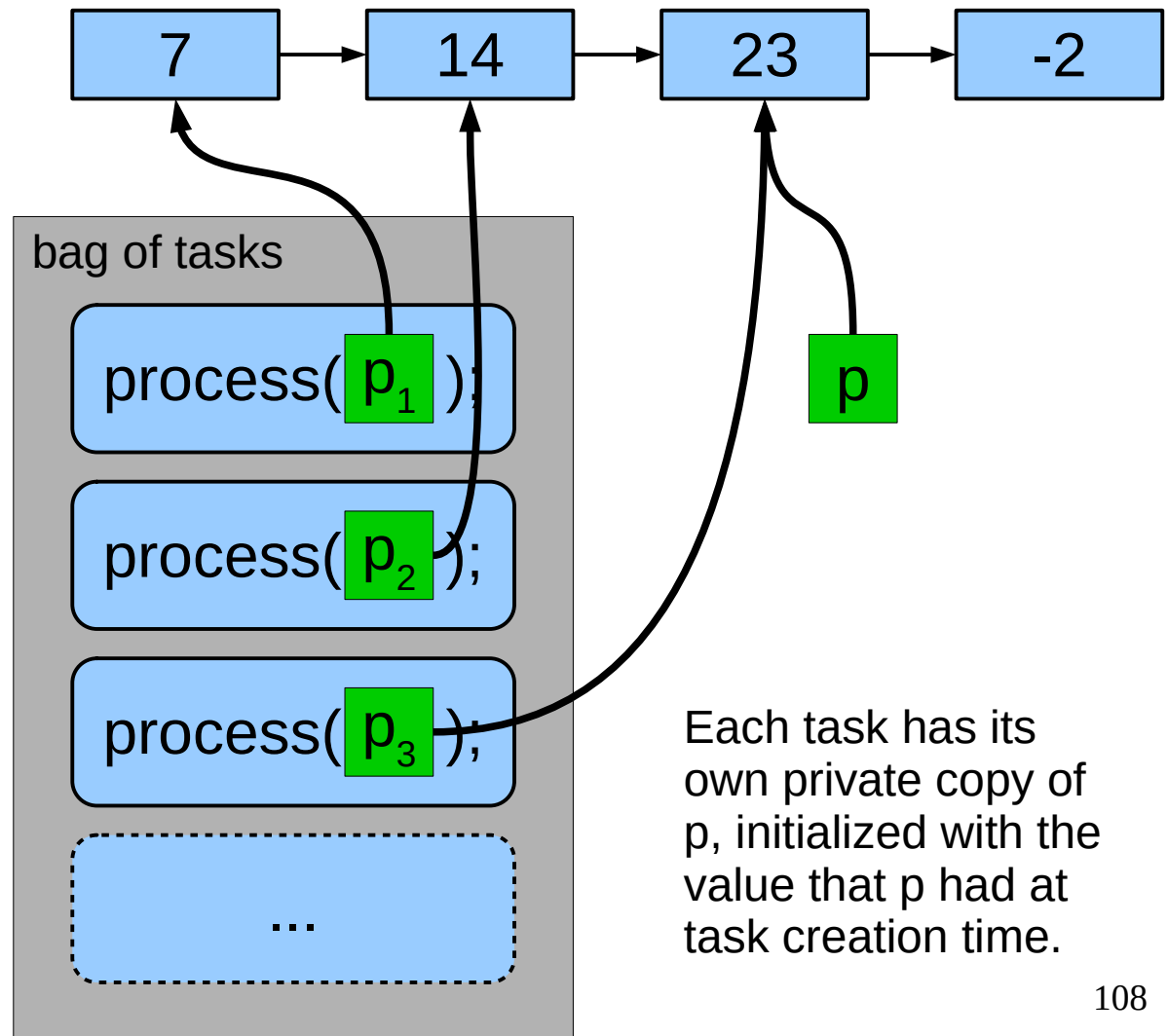


Example:

Linked list traversal with tasks

```
#pragma omp parallel
{
  #pragma omp single
  {
    ListNode *p = head;
    while (p) {
      #pragma omp task
      process(p);
      p = p->next;
    }
  }
}
```

CORRECT



When/Where are tasks completed?

- At thread barriers (explicit or implicit)
 - e.g., at the end of a `#pragma omp parallel` block
 - applies to all tasks generated in the current parallel region up to the barrier
- At `taskwait` directive
 - wait until all tasks defined `in the current task` (not in the descendants) have completed.
 - `#pragma omp taskwait`
 - The code executed by a thread in a parallel region is considered a task here

Example: parallel Fibonacci with tasks

```
#include <stdio.h>
int fib( int n )
{
    int n1, n2;
    if (n < 2) {
        return 1;
    } else {
        n1 = fib(n-1);
        n2 = fib(n-2);
        return n1 + n2;
    }
}

int main( int argc, char* argv[] )
{
    int n = 10, res;
    res = fib(n);
    printf("fib(%d)=%d\n", n, res);
    return 0;
}
```

- $F_n = F_{n-1} + F_{n-2}$
– $F_0 = F_1 = 1$
- Inefficient algorithm $O(2^n)$

Example: parallel Fibonacci with tasks

```
#include <stdio.h>
int fib( int n )
{
    int n1, n2;
    if (n < 2) {
        return 1;
    } else {
#pragma omp task shared(n1)
        n1 = fib(n-1);
#pragma omp task shared(n2)
        n2 = fib(n-2);
#pragma omp taskwait
        return n1 + n2;
    }
}

int main( int argc, char* argv[] )
{
    int n = 10, res;
#pragma omp parallel
#pragma omp master
    res = fib(n);
    printf("fib(%d)=%d\n", n, res);
    return 0;
}
```

- Binary tree of tasks
- A task cannot complete until all tasks below it in the tree are complete
 - enforced with **taskwait**
- **n1, n2** are private to the current task
 - because they are local to a function called in a parallel block
 - So they would be **firstprivate** by default
- Must be **shared** on child tasks because their value must be stored at their parent

Concluding Remarks

- OpenMP is a standard for programming shared-memory systems.
 - Uses both **special functions** and **preprocessor directives** called *pragmas*.
- OpenMP programs start multiple threads
 - By default most systems use a **block-partitioning** of the iterations in a parallelized for loop
 - OpenMP offers a variety of **scheduling options**.
- In OpenMP the **scope** of a variable is the collection of threads to which the variable is accessible.
- A **reduction** is a computation that repeatedly applies the same reduction operator to a sequence of operands in order to get a single result.